FORMAÇÃO · MÓDULO 1

FreeRTOS

Fundamentos de sistemas operativos de tempo real

Rúben Marques · 24 de setembro de 2026

PLANO DA FORMAÇÃO

Módulos e sessões práticas, pela ordem da formação

01	Módulo 1 · Fundamentos de sistemas operativos de tempo real	manhã	dia 1
02	Módulo 2 · Gestão de memória	manhã	dia 1
03	Módulo 3 · Gestão de tasks	manhã	dia 1
04	Módulo 4 · Filas (queues)	manhã	dia 1
P1	Prática 1 · Ambiente e primeiro projeto	tarde	dia 1
P2	Prática 2 · Preempção e prioridades EXTRA	tarde	dia 1
05	Módulo 5 · Gestão de interrupções	manhã	dia 2
06	Módulo 6 · Gestão de recursos	manhã	dia 2
07	Módulo 7 · Software timers	manhã	dia 2
08	Módulo 8 · Troubleshooting e debug	manhã	dia 2
P3	Prática 3 · Interrupções, recursos, timers e debug	tarde	dia 2
P4	Prática 4 · Aprofundar: ISRs, recursos, timers e debug EXTRA	tarde	dia 2

INTRODUÇÃO

O que é um RTOS

Um sistema operativo pequeno e determinístico.

Pensado para sistemas embebidos que têm de reagir a eventos externos dentro de restrições temporais estritas.

Onde se usa

Dispositivos médicos, ECUs automóveis e outros sistemas embebidos.

Mais leve que um SO de uso geral

Adequado a dispositivos com memória, CPU e energia limitadas.

Útil mesmo sem hard real-time

Muitas vezes só 1 ou 2 requisitos exigem timing determinístico.

Fonte: freertos.org — RTOS Fundamentals

MULTITASKING

Kernel, threads e tasks

O **kernel** é o componente central do SO. Se consegue executar várias threads aparentemente em simultâneo, o SO é **multitasking**.

SO DE USO GERAL · EX. LINUX

Processos e threads

- Vários utilizadores partilham o processador
- Cada programa = um ou mais threads
- Memória virtual separa processos

RTOS PEQUENO · EX. FREERTOS

Tasks

- Sem memória virtual
- Logo, sem distinção processo / thread
- Aos threads chama-se tasks

Fonte: freertos.org — RTOS Fundamentals

MULTITASKING

Porque simplifica o design

Partir a aplicação

Multitasking e comunicação entre tasks permitem dividir uma aplicação complexa em tasks mais pequenas e geríveis.

Testar, dividir, reutilizar

A partição facilita os testes, a divisão do trabalho entre equipas e a reutilização de código.

Timing no kernel

Os detalhes de timing e sequenciação passam a ser responsabilidade do kernel, e não do código da aplicação.

Fonte: freertos.org — RTOS Fundamentals

MULTITASKING

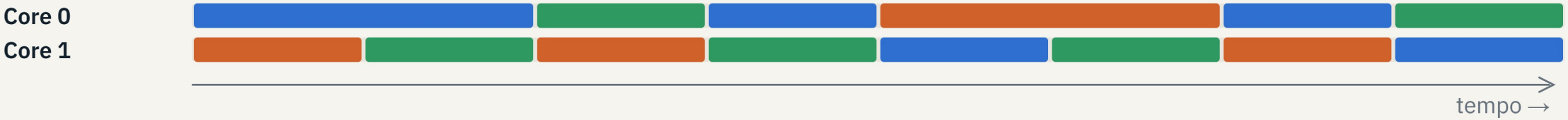
Multitasking vs. concorrência

Um core executa uma task de cada vez; ao trocar rapidamente entre elas, o SO faz parecer que correm em simultâneo. Com dois cores, duas correm de facto.

O que parece: execução concorrente

O que acontece num único core: uma task de cada vez

Com dois cores (ex.: RP2350): duas tasks correm mesmo ao mesmo tempo



Fonte: freertos.org — RTOS Fundamentals (alterado)

SCHEDULING

Quem decide o que corre

`Scheduler`	A parte do kernel que decide **que task executa em cada instante**. Pode pausar e retomar uma task muitas vezes durante a sua vida.
`Swap out / in`	Se a task B substitui a task A, A foi **swapped out** (pausada) e B **swapped in** (retomada).
`Scheduling policy`	O algoritmo usado para decidir. Num SO multi-utilizador procura dar a cada task uma fatia *justa* de CPU; num sistema real-time o objetivo é outro (já a seguir).

Fonte: freertos.org — RTOS Fundamentals

SCHEDULING

Quando uma task sai do CPU

Uma task só é swapped out quando o scheduling algorithm decide executar outra. Isso acontece de duas formas:

SEM A TASK SABER

Fica pronta uma task de prioridade mais alta

- No tick: acaba um delay ou um timeout
- Numa interrupção: a ISR dá um semáforo ou envia para uma queue
- Noutra task: uma chamada à API acorda-a

Com time slicing, o tick também troca entre tasks da mesma prioridade.

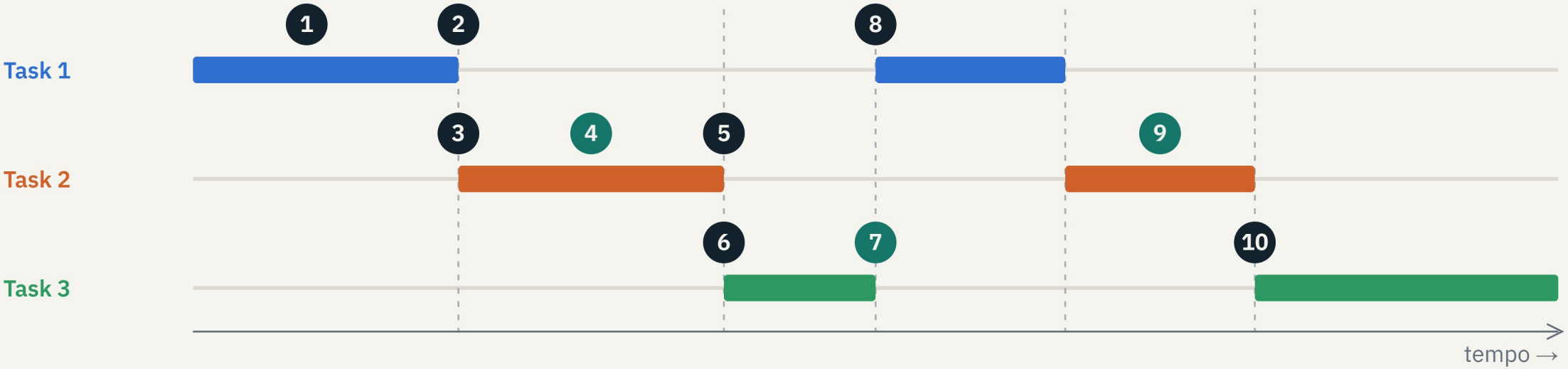
A PRÓPRIA TASK CHAMA A API

Yield	Cede o CPU; o scheduler pode voltar a escolher a mesma task.
Sleep / delay	Fica indisponível até passar o período de atraso indicado.
Block	Fica indisponível até ocorrer um evento (ex.: dados na UART) ou expirar um timeout.

Fonte: freertos.org — RTOS Fundamentals; Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

SCHEDULING

Exemplo: três tasks e um periférico



1 Task 1 a executar

2 O kernel faz swap out da task 1…

3 …e swap in da task 2

4 Task 2 bloqueia (lock) um periférico

5 Swap out da task 2…

6 …e swap in da task 3

7 Task 3 encontra o periférico ocupado e bloqueia

8 Swap in da task 1 (etc.)

9 Task 2 termina e liberta o periférico

10 Task 3 encontra-o livre e continua

Fonte: freertos.org — RTOS Fundamentals

REAL-TIME SCHEDULING

Mesmos princípios, outro objetivo

Os eventos do mundo real têm **deadlines**. A scheduling policy de um RTOS tem de garantir que são cumpridos.

01

Cada task tem uma prioridade

Atribuída pelo engenheiro de software.

02

Corre a mais prioritária

Recebe CPU a task de maior prioridade que está apta a executar.

03

Opcional: partilha entre iguais

Tempo dividido de forma justa entre tasks da mesma prioridade, se estiverem prontas.

⚠ **Não é magia:** não abranda nem acelera o tempo. Cabe a quem escreve a aplicação garantir que as restrições temporais são exequíveis com as prioridades escolhidas.

Fonte: freertos.org — RTOS Fundamentals

Exemplo: teclado, LCD e controlo

Teclado e LCD

Soft real-time | resposta ≤ 100 ms

```
void vKeyHandlerTask( void *pvParameters )
{
  // Processo contínuo: loop infinito
  for( ;; )
  {
    [Bloquear à espera de uma tecla]
    [Processar a tecla]
  }
}
```

Controlo com filtro digital

Ciclo de 2 ms | precisão ± 0,5 ms

```
void vControlTask( void *pvParameters )
{
  for( ;; )
  {
    [Esperar até 2 ms após o ciclo anterior]
    [Amostrar a entrada]
    [Filtrar a amostra]
    [Executar o algoritmo de controlo]
    [Escrever o resultado]
  }
}
```

Fonte: freertos.org — RTOS Fundamentals

REAL-TIME SCHEDULING

Quem fica com a prioridade mais alta

A task de controlo recebe a prioridade mais alta porque:

1 **O prazo é mais apertado**

O deadline do controlo é mais restrito do que o do tratamento do teclado.

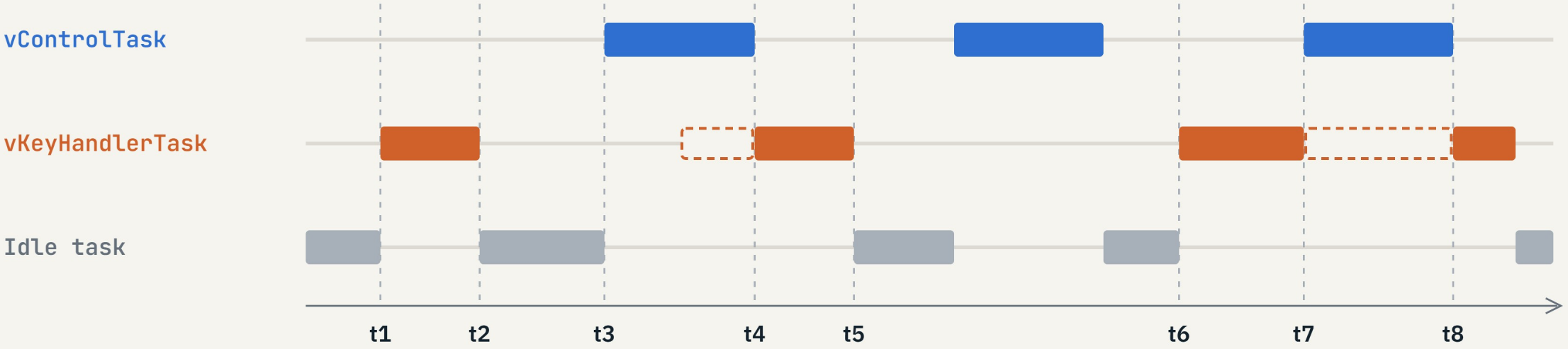
2 **Falhar custa mais**

As consequências de um deadline falhado são maiores no controlo do que no teclado.

Fonte: freertos.org — RTOS Fundamentals

REAL-TIME SCHEDULING

O exemplo ao longo do tempo



Entre t3 e t4

Há uma tecla premida (tracejado), mas vControlTask tem maior prioridade: a tecla espera até t4.

t7: preempção

O timer dispara a meio do processamento da tecla. vKeyHandlerTask para e só termina depois de t8.

Sem nada pronto

O CPU vai para a idle task, que está sempre apta a executar.

Fonte: freertos.org — RTOS Fundamentals

EM RESUMO

Um RTOS dá sempre o CPU à task de maior prioridade que está pronta a executar.

Task | Scheduler | Swap in / out | Yield · delay · block | Prioridade | Idle task

A seguir, no Módulo 2: gestão de memória (heap).

FORMAÇÃO · MÓDULO 2

Gestão de memória

Heap memory management: de onde vem a RAM das tasks e dos objetos do kernel

ESTÁTICA OU DINÂMICA

pvPortMalloc() e vPortFree(), e porque não o malloc() da biblioteca C

HEAP_1 A HEAP_5

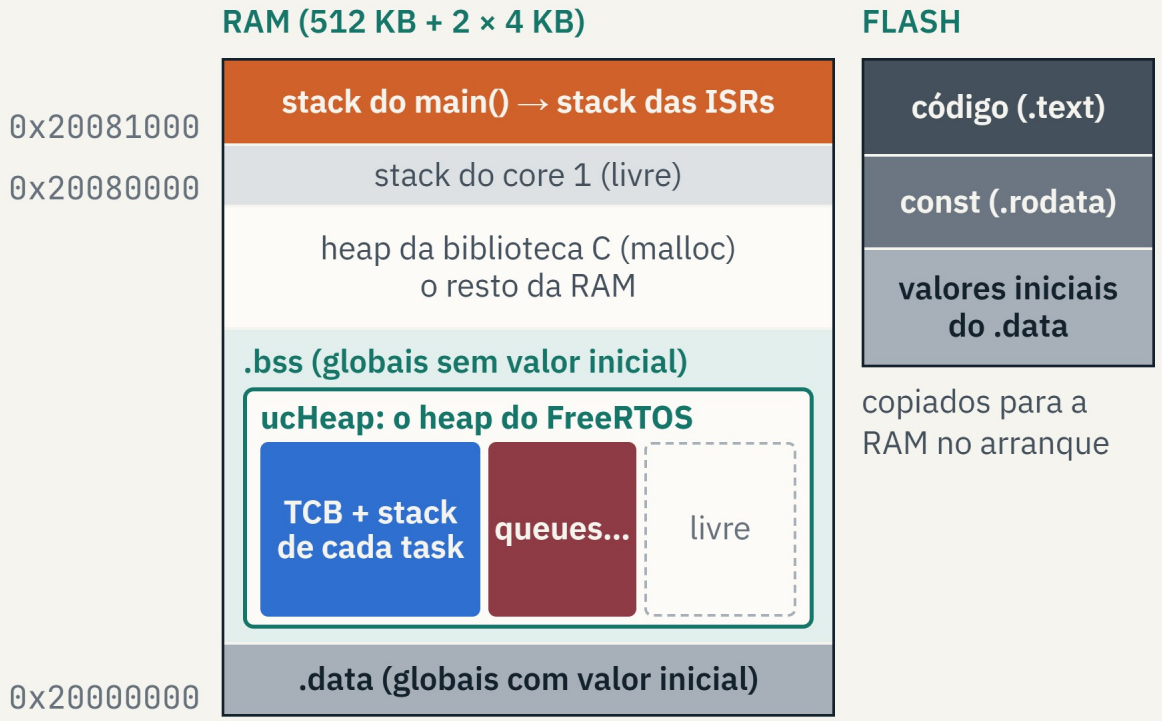
Os cinco esquemas, fragmentação, best fit e first fit

DIAGNÓSTICO

Medir o heap, malloc failed hook, alocação estática

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 3

Onde fica cada coisa na memória (Pico 2)




```c
int iCount = 5; /* .data */
int iErros; /* .bss */
const char pcMsg[] = "ok"; /* flash */

void vTask( void *pv )
{
int iLocal; /* stack da task */
static int iN; /* .bss */
void *p = pvPortMalloc( 64 ); /* ucHeap */
void *q = malloc( 64 ); /* heap C */
}
```

> As globais **nunca** vão para o heap. O heap do FreeRTOS é ele próprio uma global: um array no .bss.

Fonte: linker script do pico-sdk e port RP2350_RISC-V; fora de escala

GESTÃO DO HEAP

Alocação estática ou dinâmica

A RAM dos objetos do kernel (tasks, queues, semáforos, event groups…) pode ser reservada em **compile time** ou em **run time**.

configSUPPORT_DYNAMIC_ALLOCATION = 1

Dinâmica

- Menos esforço de design e planeamento
- API mais simples
- Minimiza o footprint de RAM

configSUPPORT_STATIC_ALLOCATION = 1

Estática

- Mais determinística
- Não há falhas de alocação para tratar
- Sem risco de fragmentação do heap

As duas podem estar ativas ao mesmo tempo. A dinâmica está ativa por omissão (constante indefinida).

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

Porque não usar malloc() e free()

A alocação dinâmica é um conceito de C, não do FreeRTOS. Mas as funções da biblioteca standard podem não servir:

Nem sempre existem
em sistemas embebidos pequenos

Ocupam muito código
implementação relativamente grande

Raramente thread-safe
perigosas com várias tasks

Não determinísticas
o tempo de execução varia de chamada para chamada

Fragmentação
memória livre suficiente, mas não contígua

Complicam o linker
configuração mais difícil

Erros difíceis de depurar
se o heap crescer para cima de outras variáveis

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

pvPortMalloc() e vPortFree()

A alocação de memória faz parte da **portable layer**, não do núcleo do kernel: cada sistema tem requisitos diferentes e nenhum algoritmo serve para todos.

Quando precisa de RAM, o FreeRTOS chama **pvPortMalloc()** em vez de malloc(); para libertar, **vPortFree()** em vez de free(). Mesmos protótipos, e são públicas: a aplicação também as pode usar.

O FreeRTOS traz **5 implementações de exemplo**. A aplicação usa uma delas ou fornece a sua.

PROTÓTIPOS

```
void *pvPortMalloc( size_t xWantedSize );
void vPortFree( void *pv );
```

ONDE ESTÃO

```
FreeRTOS/Source/portable/MemMang/
  heap_1.c heap_2.c heap_3.c
  heap_4.c heap_5.c
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

Os cinco esquemas de exemplo

Esquema	Algoritmo	Liberta	Junta livres	Determinístico	Quando usar
heap_1	Só subdivide o array	Não	—	Sim	Objetos nunca apagados
heap_2	Best fit	Sim	Não	Não	Legado: usar heap_4
heap_3	malloc()/free() da libc	Sim	(libc)	Não	Heap do linker; suspende o scheduler
heap_4	First fit	Sim	Sim	Não	Alocar e libertar tamanhos variados
heap_5	First fit (como heap_4)	Sim	Sim	Não	RAM em várias regiões não contíguas

heap_1, heap_2 e heap_4 subdividem um array estático de **configTOTAL_HEAP_SIZE** bytes. Por isso o FreeRTOS parece gastar muita RAM: o heap conta como dados do FreeRTOS.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

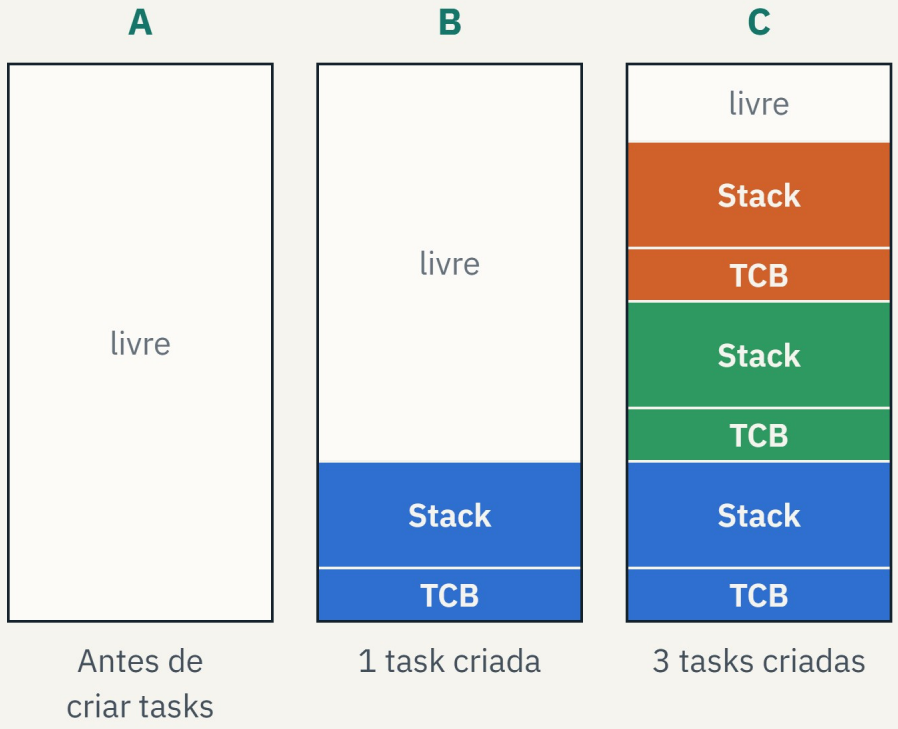
heap_1: só aloca, nunca liberta

Muitos sistemas pequenos criam todas as tasks e objetos **antes** de arrancar o scheduler, e nunca os apagam.

heap_1 implementa só **pvPortMalloc()**: vai subdividindo um array **uint8_t** de **configTOTAL_HEAP_SIZE** bytes. Não há **vPortFree()**.

Cada task criada dinamicamente = **2 alocações**: o TCB e a stack.

Sempre determinístico e nunca fragmenta: candidato até em sistemas safety-critical.



Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

heap_2 e heap_4: best fit vs. first fit

heap_2 · legado

Best fit: o bloco de tamanho mais próximo

Blocos livres, por ordem de endereço: 5, 200, 25, 100 bytes.
Pedido: **20 bytes**.

Percorre todos e escolhe o mais pequeno que chega: o de 25. Fica 20 + 5 livres.

Não junta blocos livres adjacentes, logo fragmenta mais. Não fragmenta se os blocos forem sempre do mesmo tamanho.

heap_4 · recomendado

First fit: o primeiro bloco que chega

Blocos livres, por ordem de endereço: 5, 200, 25, 100 bytes.
Pedido: **20 bytes**.

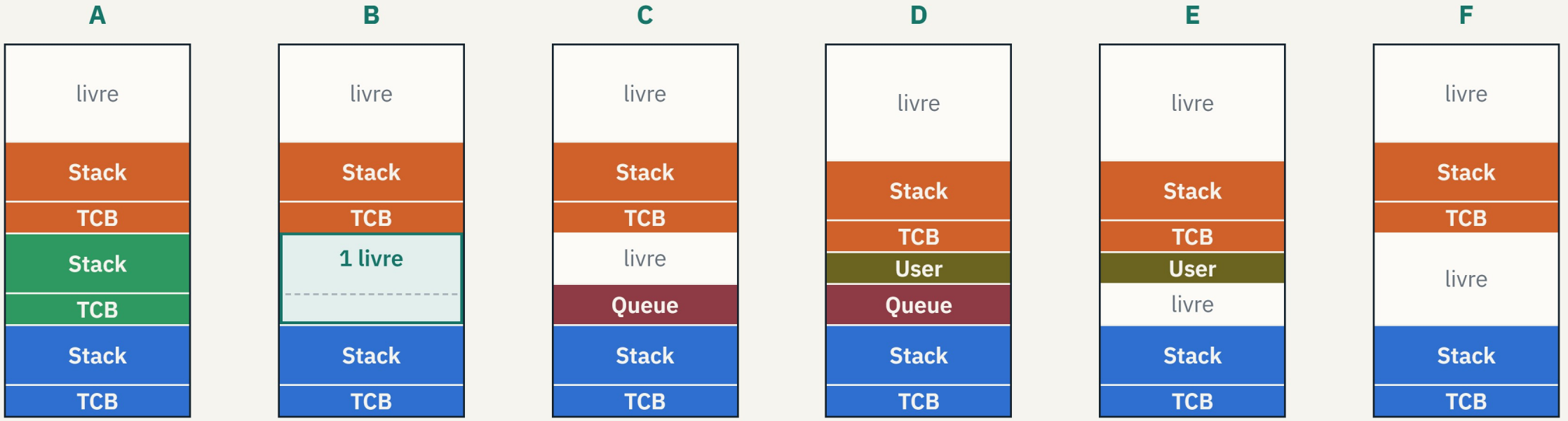
Escolhe o primeiro que chega: o de 200, sem olhar para os outros. Fica 20 + 180 livres.

Junta (coalesce) blocos livres adjacentes, o que minimiza a fragmentação.

Simplificação: os dois guardam o tamanho de cada bloco dentro do heap, por isso as partes somam um pouco menos do que o bloco original.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3; exemplo adaptado

heap_4: first fit com coalescência



A Três tasks criadas

B Task apagada: TCB e stack eram 2 blocos e juntam-se num só livre (no heap_2 ficavam 2)

C xQueueCreate(): first fit usa esse bloco e parte-o

D pvPortMalloc() da aplicação cabe no espaço seguinte

E Queue apagada: livre dos dois lados do bloco User

F vPortFree() do User: os três livres juntam-se num só

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

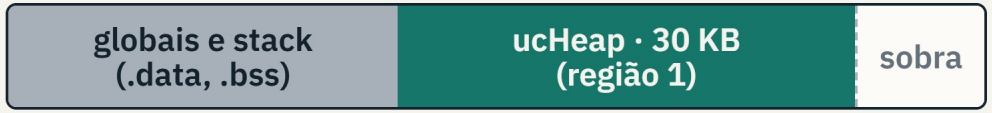
heap_5: um heap feito de várias regiões

Mesmo algoritmo do heap_4, mas junta **várias regiões de RAM separadas** num só heap. Útil quando a RAM não aparece como um bloco contíguo no mapa de memória.

É o único que exige inicialização explícita: nenhum objeto do kernel pode ser criado dinamicamente antes de **vPortDefineHeapRegions()**.

Regras do array: regiões ordenadas por endereço crescente, terminado com uma entrada **{ NULL, 0 }**.

RAM1 (64 KB): quem a arruma é o linker



Truque: o **ucHeap** é uma variável global. O linker escolhe o endereço, não há sobreposição e, se não couber, o link falha.

```
#define RAM2_START_ADDRESS ( ( uint8_t * ) 0x00020000 )
#define RAM2_SIZE ( 32 * 1024 )
#define RAM3_START_ADDRESS ( ( uint8_t * ) 0x00030000 )
#define RAM3_SIZE ( 32 * 1024 )
#define RAM1_HEAP_SIZE ( 30 * 1024 )
static uint8_t ucHeap[ RAM1_HEAP_SIZE ]; /* na RAM1 */
const HeapRegion_t xHeapRegions[] =
{
    { ucHeap, RAM1_HEAP_SIZE },
    { RAM2_START_ADDRESS, RAM2_SIZE },
    { RAM3_START_ADDRESS, RAM3_SIZE },
    { NULL, 0 }
};
vPortDefineHeapRegions( xHeapRegions );
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

Medir e vigiar o heap

Função / macro	O que dá	Disponível em
xPortGetFreeHeapSize()	Bytes livres neste momento. Não diz nada sobre fragmentação.	Todos exceto heap_3
xPortGetMinimumEverFreeHeapSize()	Mínimo de bytes livres desde o arranque: quão perto esteve de ficar sem heap. Ajuda a reduzir configTOTAL_HEAP_SIZE.	heap_4, heap_5
vPortGetHeapStats()	Preenche um HeapStats_t: total livre, maior e menor bloco livre, nº de blocos, mínimo histórico, nº de alocações e libertações.	heap_4, heap_5
traceMALLOC / traceFREE	Macros de trace para recolher estatísticas de heap por task.	Aplicação
vApplicationMallocFailedHook()	Chamada quando pvPortMalloc() devolve NULL (configUSE_MALLOC_FAILED_HOOK = 1). O objeto não é criado.	Aplicação

Exemplo: se o mínimo histórico for 2000 bytes depois de correr o código com mais uso de heap, configTOTAL_HEAP_SIZE pode ser reduzido em até 2000 bytes.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3

GESTÃO DO HEAP

Alocação estática

Com **configSUPPORT_STATIC_ALLOCATION = 1**, o programador cria explicitamente cada bloco de memória:

Toda a memória conhecida em compile time

Toda a memória determinística

Ficam disponíveis as versões **...Static()**: xTaskCreateStatic, xQueueGenericCreateStatic, xQueueCreateMutexStatic, xTimerCreateStatic, xEventGroupCreateStatic, xStreamBufferGenericCreateStatic...

Custo: a aplicação fornece a memória da idle task (e da timer task, se configUSE_TIMERS = 1), e os buffers têm de ter scope adequado: **static**.

Implementação típica (Listing 3.15)

```
void vApplicationGetIdleTaskMemory(
        StaticTask_t **ppxIdleTaskTCBBuffer,
        StackType_t **ppxIdleTaskStackBuffer,
        uint32_t *pulIdleTaskStackSize )
{
    static StaticTask_t xIdleTaskTCB;
    static StackType_t
        uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];
    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

Na Pico 2: com o target CMake **FreeRTOS-Kernel-Static**, o kernel já fornece esta memória (configKERNEL_PROVIDED_STATIC_MEMORY = 1).

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 3; port RP2350

MÓDULO 2 EM RESUMO

A RAM das tasks e dos objetos do kernel vem do heap do FreeRTOS ou de memória estática; para a maioria dos casos, heap_4.

pvPortMalloc / vPortFree

heap_1 ... heap_5

configTOTAL_HEAP_SIZE

Fragmentação

xPortGetFreeHeapSize

Malloc failed hook

Estática vs. dinâmica

A seguir, no Módulo 3: gestão de tasks.

FORMAÇÃO · MÓDULO 3

Gestão de tasks

Criar tasks, prioridades, estados e algoritmos de scheduling

CRIAR TASKS

Task functions, xTaskCreate(), parâmetros e prioridades

ESTADOS E TEMPO

O tick, Blocked, vTaskDelay() e vTaskDelayUntil(), a idle task

SCHEDULING

Preemptivo com e sem time slicing, e cooperativo

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 4

GESTÃO DE TASKS

Uma task é uma função C

```
void vATaskFunction( void *pvParameters )
{
    /* Uma cópia por instância, na stack da task */
    long lStackVariable = 0;

    /* Uma só cópia, partilhada por todas as instâncias */
    static long lStaticVariable = 0;

    for( ;; )
    {
        /* Funcionalidade da task */
    }

    /* Se sair do loop: apagar antes do fim da função */
    vTaskDelete( NULL );
}
```

Nunca retorna

Sem return e sem chegar ao fim da função. Se já não é precisa, apaga-se com vTaskDelete().

Normalmente um loop infinito

Cada task é um pequeno programa com o seu entry point.

Uma função, várias instâncias

Cada task criada tem a sua própria stack e as suas variáveis automáticas.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

Seis funções para criar tasks

Cada task precisa de **dois blocos de RAM**: o **TCB** (Task Control Block) e a **stack**. O nome da função diz de onde vem essa RAM e com que privilégios corre a task.

`xTaskCreate()` `xTaskCreateStatic()` `xTaskCreateRestricted()` `xTaskCreateRestrictedStatic()`
`xTaskCreateAffinitySet()` `xTaskCreateStaticAffinitySet()`

...Static

RAM pré-alocada

Os blocos são passados como parâmetros. Sem Static, a RAM vem do heap em run time.

...Restricted

Acesso limitado à memória

Task em modo não privilegiado (ports que o suportam). Sem Restricted, corre em modo privilegiado com acesso a toda a memória.

Não disponível na Pico 2

...AffinitySet

Escolher o core

Em ports SMP, define em que core(s) a task pode correr.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

xTaskCreate()

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,
    const char * const pcName,
    configSTACK_DEPTH_TYPE usStackDepth,
    void *pvParameters,
    UBaseType_t uxPriority,
    TaskHandle_t *pxCreatedTask );
```

> **configSTACK_DEPTH_TYPE** é uint16_t por omissão. Se profundidade × largura da stack passar de 65535, defini-lo como size_t ou unsigned long.

pvTaskCode	Ponteiro para a função da task.
pcName	Nome descritivo, só para debug.
usStackDepth	Stack em words, não bytes: 128 × 4 B = 512 B.
pvParameters	Valor passado ao parâmetro da task.
uxPriority	0 (mais baixa) a configMAX_PRIORITIES – 1.
pxCreatedTask	Recebe o handle; NULL se não for preciso.
Retorno	pdPASS, ou pdFAIL se faltar heap.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

Exemplo: duas instâncias da mesma task

A TASK

```
void vTaskFunction( void *pvParameters )
{
    char *pcTaskName;
    pcTaskName = ( char * ) pvParameters;

    for( ;; )
    {
        vPrintLine( pcTaskName );
        /* atraso grosseiro: null loop */
    }
}
```

Mesma prioridade: as duas alternam no CPU

Strings static const: continuam válidas depois de o main() deixar de correr

O main()

```
static const char *pcTextForTask1 =
    "Task 1 is running";
static const char *pcTextForTask2 =
    "Task 2 is running";

int main( void )
{
    xTaskCreate( vTaskFunction, "Task 1", 1000,
        ( void * ) pcTextForTask1, 1, NULL );
    xTaskCreate( vTaskFunction, "Task 2", 1000,
        ( void * ) pcTextForTask2, 1, NULL );
    vTaskStartScheduler();
    /* Só chega aqui se faltar heap */
    for( ;; );
}
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

Prioridades

O scheduler garante sempre que entra em **Running** a task de maior prioridade que pode correr. Tasks de igual prioridade entram e saem de Running à vez.

Prioridade inicial em **uxPriority**; muda-se depois com **vTaskPrioritySet()**. Qualquer número de tasks pode partilhar a mesma prioridade.

Manter **configMAX_PRIORITIES** o mais baixo possível: mais prioridades gastam mais RAM.

`configMAX_PRIORITIES - 1`
A mais alta

⋮

`0`
A mais baixa (a da idle task)

Scheduler genérico
Em C, todos os ports. Sem limite de prioridades; pior caso cresce com o número.

Otimizado
Em assembly. Máx. 32 (32 bits) ou 64 (64 bits); pior caso constante.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

O tick interrupt e a medição do tempo

`configTICK_RATE_HZ = 100`

10 ms

por tick period = um time slice

Um interrupt periódico, o **tick**, corre o scheduler no fim de cada time slice para escolher a próxima task.

> **O fim do time slice não é o único ponto de decisão**
>
> O scheduler também decide logo que:
>
> • a task em Running bloqueia (delay, queue vazia, semáforo...)
>
> • uma ISR põe em Ready uma task de maior prioridade
>
> • outra task a acorda, ou lhe sobe a prioridade

A API mede o tempo em ticks. **pdMS_TO_TICKS()** converte milissegundos: o código não muda se a frequência do tick mudar.

```
TickType_t xTimeInTicks = pdMS_TO_TICKS( 200 );
```

O **tick count** conta os ticks desde o arranque do scheduler; o FreeRTOS trata do overflow.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

Starvation e tasks event-driven

EXEMPLO A · NULL LOOP

Task 1 prioridade 1, Task 2 prioridade 2

```
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
Task 2 is running
```

A Task 2 pode correr sempre. A Task 1 nunca entra em Running: sofre **starvation**.

Exemplo B · vTaskDelay()

Mesmas prioridades, agora a bloquear

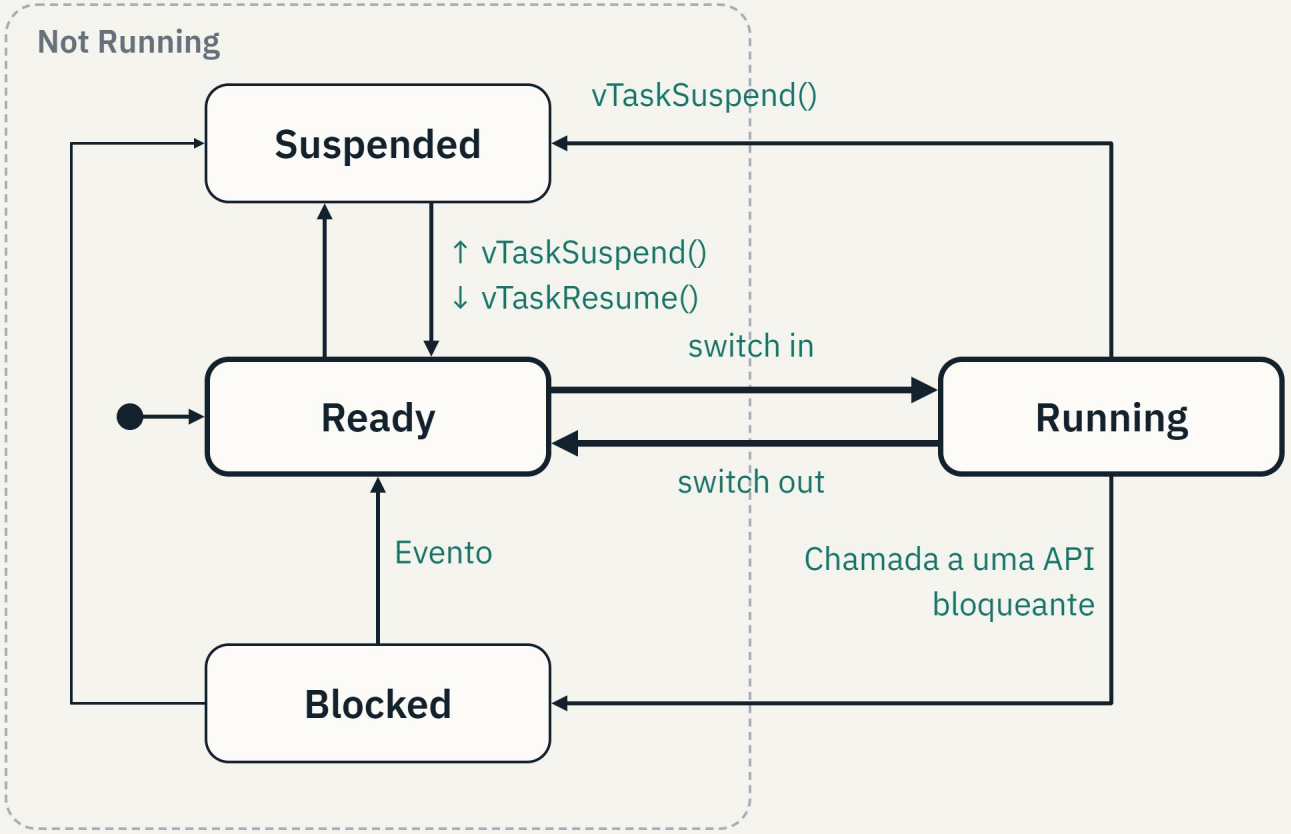
```
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
```

Enquanto está **Blocked**, a task não gasta CPU. As duas correm; no resto do tempo corre a idle task.

Tasks de processamento contínuo só servem na prioridade mais baixa. Uma task **event-driven** só entra em Running depois de um evento a acordar.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

Os estados de uma task



Running
O processador está a executar esta task. Num single core, só uma de cada vez.

Ready
Pode correr, mas não está a correr: nem Blocked nem Suspended.

Blocked
À espera de um evento temporal ou de sincronização.

Suspended
Invisível para o scheduler. Só entra com vTaskSuspend() (de qualquer estado) e só sai com vTaskResume() ou xTaskResumeFromISR(). A maioria das aplicações não o usa.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

O estado Blocked

Uma task em Blocked está à espera de um de dois tipos de evento:

Eventos temporais

Expira um período de atraso ou chega-se a um tempo absoluto. Ex.: esperar 10 ms.

Usados para comportamento periódico ou timeouts.

Eventos de sincronização

Vêm de outra task ou de um interrupt. Ex.: esperar que cheguem dados a uma queue.

Queues, semáforos, mutexes, event groups, stream e message buffers, task notifications.

Os dois ao mesmo tempo: bloquear numa queue com timeout de 10 ms. A task sai de Blocked quando chegam dados ou quando passam os 10 ms sem dados.

GESTÃO DE TASKS

vTaskDelay() e vTaskDelayUntil()

Relativo · INCLUDE_vTaskDelay = 1

vTaskDelay(xTicksToDelay)

```
const TickType_t xDelay250ms =
    pdMS_TO_TICKS( 250 );
for( ;; )
{
    vPrintLine( pcTaskName );
    vTaskDelay( xDelay250ms );
}
```

Blocked durante N ticks **a partir da chamada**. Chamada no tick 10 000 com 100: volta a Ready no tick 10 100.

ABSOLUTO

vTaskDelayUntil(&xLastWake, xInc)

```
TickType_t xLastWakeTime = xTaskGetTickCount();
for( ;; )
{
    vPrintLine( pcTaskName );
    vTaskDelayUntil( &xLastWakeTime,
                     pdMS_TO_TICKS( 250 ) );
}
```

Acorda num tick count exato, calculado a partir do último despertar. É a escolha para **períodos fixos**.

xLastWakeTime inicializa-se uma vez; depois é atualizado pela própria função.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

GESTÃO DE TASKS

A idle task e o idle hook

IDLE TASK

Criada automaticamente por **vTaskStartScheduler()**: tem de haver sempre pelo menos uma task que possa correr.

Prioridade 0: é preemptada logo que outra task fica Ready. **configIDLE_SHOULD_YIELD** controla a partilha com tasks da aplicação na mesma prioridade.

> É a idle task que liberta a memória das tasks que se apagaram a si próprias. **Não a deixar sem CPU.**

Idle hook · configUSE_IDLE_HOOK = 1

```
void vApplicationIdleHook( void );
```

Chamado uma vez por iteração do loop da idle task.
Usos comuns:

- Processamento de fundo, sem a RAM de uma task extra
- Medir a capacidade de processamento livre
- Pôr o processador em modo de baixo consumo

Regras: nunca bloquear nem suspender; retornar em tempo razoável.

GESTÃO DE TASKS

Mudar a prioridade e apagar tasks

INCLUDE_vTaskPrioritySet / INCLUDE_uxTaskPriorityGet = 1

Prioridade em run time

```
void vTaskPrioritySet( TaskHandle_t xTask,
        UBaseType_t uxNewPriority );
UBaseType_t uxTaskPriorityGet(
        TaskHandle_t xTask );
```

NULL como handle = a própria task. O handle só é preciso para mexer noutra task (guarda-se no pxCreatedTask).

INCLUDE_vTaskDelete = 1

Apagar uma task

```
void vTaskDelete( TaskHandle_t xTaskToDelete );
```

NULL = apaga-se a si própria. A task deixa de existir e não volta a Running.

Só a memória alocada **pelo kernel** (TCB, stack) é libertada, pela idle task. O que a task alocou tem de ser libertado explicitamente.

Criar e apagar tasks continuamente não é boa prática: preferir reutilizar tasks.

Escolher o algoritmo de scheduling

Algoritmo	Preempção	configUSE_PREEMPTION	configUSE_TIME_SLICING
Preemptivo com time slicing	Sim	1	1
Preemptivo sem time slicing	Sim	1	0
Cooperativo	Não	0	Qualquer

Fixed priority

O algoritmo não muda as prioridades, mas as tasks podem mudá-las.

Preemptive

Se fica Ready uma task de maior prioridade, a que está em Running sai de imediato, sem yield nem block.

Time slicing

Tasks de igual prioridade trocam no fim de cada time slice, isto é, a cada tick.

Em qualquer configuração single core, tasks da mesma prioridade são escolhidas à vez (**round robin**), o que não garante tempo igual.

SMP Com vários cores (ex.: os 2 do RP2350), as mesmas regras escolhem uma task **por core**: correm ao mesmo tempo as tasks Ready de maior prioridade. Com **configRUN_MULTIPLE_PRIORITIES = 1**, podem ser de prioridades diferentes.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

SCHEDULING ALGORITHMS

Com ou sem time slicing

configUSE_TIME_SLICING = 1

Troca a cada tick

Com várias tasks Ready na prioridade mais alta, o scheduler escolhe outra em cada tick interrupt.

É o comportamento por omissão da maioria das aplicações.

configUSE_TIME_SLICING = 0

Só troca quando é obrigado

Quando fica Ready uma task de maior prioridade, ou quando a task em Running passa a Blocked ou Suspended.

Menos context switches e menos overhead, mas tasks de igual prioridade podem receber tempos muito diferentes.

Técnica avançada: só para utilizadores experientes.

`configIDLE_SHOULD_YIELD`	A 1, a idle task cede o resto do time slice se houver outras tasks de prioridade 0 em Ready. A 0, fica o time slice inteiro.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4

SCHEDULING ALGORITHMS

Scheduling cooperativo

Com **configUSE_PREEMPTION = 0**, só há troca de contexto quando a task em Running bloqueia ou chama **taskYIELD()**. Nunca há preempção, logo também não há time slicing.

COM PREEMPÇÃO: DUAS TASKS A ESCREVER NA MESMA UART

```
abcdefg123456789hijklmnop
```

Task 1 escreve “abcdefghijklmnop”, mas é trocada a meio; a **Task 2** escreve “123456789” no meio da string.

Cooperativo: mais fácil de proteger

Somos nós que decidimos quando há troca: nunca a meio de um recurso inconsistente.

Preemptivo: mais responsivo

A task de maior prioridade corre logo que fica Ready. Muitas vezes essencial em real-time.

MÓDULO 3 EM RESUMO

Tasks são funções que nunca retornam, passam a maior parte do tempo em Blocked, e corre sempre a de maior prioridade que está pronta.

xTaskCreate · Prioridades e tick · Ready · Running · Blocked · Suspended · vTaskDelayUntil · Starvation · vTaskPrioritySet · Time slicing · Cooperativo

A seguir, no Módulo 4: filas (queues).

FORMAÇÃO · MÓDULO 4

Filas (queues)

Comunicação entre tasks

O QUE É UMA QUEUE

Os dados são copiados; comprimento e tamanho de item fixos

A API

Criar, enviar, receber, bloquear, prioridades, várias origens

PADRÕES

Ponteiros para dados grandes, mailbox

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 5

GESTÃO DE QUEUES

O que é uma queue

Mecanismo de comunicação **task → task**, **task → ISR** e **ISR → task**.

Guarda um número finito de itens de **tamanho fixo**. O comprimento (length) e o tamanho de cada item definem-se ao criar a queue.

Normalmente é uma FIFO: escreve-se no fim (tail) e lê-se da frente (head). Também se pode escrever à frente, ou sobrescrever.

Queue criada para 5 inteiros: vazia.

Task A envia x = 10 para o fim. É o único item: está no fim e na frente.

Task A muda x para 20 e envia outra vez. O 10 continua à frente.

Task B recebe para y: fica com 10, o da frente. Na queue sobra o 20.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

GESTÃO DE QUEUES

O FreeRTOS copia os dados para a queue

Queue by copy: os dados enviados são copiados byte a byte para a queue (e não apenas uma referência). É mais poderoso e mais simples:

Também serve para referências
Se o dado for grande, copia-se para a queue um ponteiro.

Variáveis da stack
Pode-se enviar uma variável local, mesmo que deixe de existir depois.

Nada para alocar antes
Não é preciso malloc() nem um array global só para poder enviar.

Pode alterar logo a variável
Logo após o xQueueSend() pode mudar: a queue tem a cópia.

Tasks desacopladas
Ninguém tem de saber quem é o dono do dado nem quem o liberta.

Memória a cargo do RTOS
O kernel aloca a memória onde os dados ficam.

Atravessa proteção de memória
Com MPU, a cópia passa fronteiras que um ponteiro não passa.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

GESTÃO DE QUEUES

Várias tasks e bloqueio numa queue

Uma queue é um objeto próprio: qualquer task ou ISR que a conheça pode escrever ou ler. É muito comum ter **vários escritores**; vários leitores é bem menos comum.

LER DE UMA QUEUE VAZIA

Bloquear à espera de dados

A task indica um block time e fica em Blocked. Passa a Ready quando outra task ou ISR escreve na queue, ou quando o tempo expira.

ESCREVER NUMA QUEUE CHEIA

Bloquear à espera de espaço

Igual: fica em Blocked até haver espaço ou até o block time expirar.

Com várias tasks bloqueadas, só uma é desbloqueada: **a de maior prioridade**. Com prioridades iguais, **a que espera há mais tempo**.

GESTÃO DE QUEUES

A API essencial

```
QueueHandle_t xQueueCreate( UBaseType_t uxQueueLength,
                            UBaseType_t uxItemSize );
BaseType_t xQueueSendToBack( QueueHandle_t xQueue,
                             const void * pvItemToQueue,
                             TickType_t xTicksToWait );
/* xQueueSendToFront(): iguais; xQueueSend() == ToBack */
BaseType_t xQueueReceive( QueueHandle_t xQueue,
                          void * const pvBuffer,
                          TickType_t xTicksToWait );
UBaseType_t uxQueueMessagesWaiting( QueueHandle_t xQueue );
```

xQueueCreate() devolve o handle, ou NULL se não houver heap.

XTICKSTOWAIT

0	Retorna logo se cheia / vazia.
pdMS_TO_TICKS(ms)	Espera no máximo esse tempo.
portMAX_DELAY	Espera sem limite (INCLUDE_vTaskSuspend = 1).

RETORNO

pdPASS, ou errQUEUE_FULL / errQUEUE_EMPTY (iguais a pdFAIL) se o tempo expirou.

> **Nunca** chamar estas funções numa ISR: usar as versões …FromISR() (módulo 5).

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

Exemplo: bloquear ao receber

2 EMISSORES (T1, T2), PRIORIDADE 1: ENVIAM 100 E 200

```
for( ;; ) /* vSenderTask */
{
    /* sem block time: nunca enche */
    xQueueSendToBack( xQueue,
        &lValueToSend, 0 );
}
```

1 RECETOR, PRIORIDADE 2

```
for( ;; ) /* vReceiverTask */
{
    /* bloqueia até 100 ms */
    if( xQueueReceive( xQueue, &lValor,
        pdMS_TO_TICKS( 100 ) ) == pdPASS )
        /* imprime lValor */
}
```

EXEMPLO EM AÇÃO · QUEUE DE 5 ITENS · ENTRA À ESQUERDA, SAI À DIREITA

1. Recetor em Blocked: queue vazia
2. T1 envia 100 → o recetor acorda e preempta-o
3. Recetor retira o 100 e volta a bloquear
4. Mais tarde a T2 envia 200 → recetor acorda
5. Retira-o: **nunca mais de 1 item**

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

GESTÃO DE QUEUES

As prioridades mudam o comportamento

EXEMPLO ANTERIOR

Recetor com prioridade mais alta

Retira cada item assim que entra: a queue **nunca tem mais de um item**. Quem envia não precisa de block time.

EXEMPLO INVERSÃO

Emissores com prioridade mais alta

Voltam a encher a queue mal há espaço: a queue está **normalmente cheia**. Quem envia bloqueia (100 ms); o recetor só corre com a queue cheia.

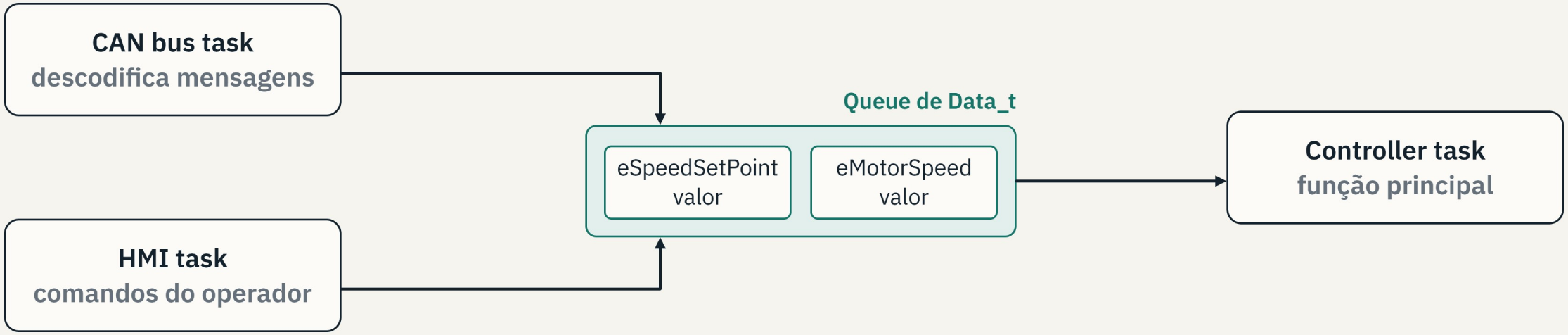
EXEMPLO INVERSÃO EM AÇÃO · QUEUE DE 3 ITENS · ENTRA À ESQUERDA, SAI À DIREITA

1. T1 envia 3 e bloqueia: **cheia**
2. T2 encontra-a cheia e bloqueia: corre o **recetor**
3. Recetor retira 1 → T1 acorda e preempta-o
4. T1 volta a encher e bloqueia
5. Recetor retira 1 → agora acorda a T2
6. T2 enche: **sempre cheia**, sai um de cada vez

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

GESTÃO DE QUEUES

Receber de várias origens numa só queue



eDataID diz ao Controller o que é o dado (velocidade do motor, novo set point...)

lDataValue leva o valor. Uma queue, uma task à espera, várias origens.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

GESTÃO DE QUEUES

Dados grandes: enviar ponteiros

Copiar dados grandes byte a byte é caro. Enviar um **ponteiro** poupa tempo e RAM, mas exige cuidado:

Dono bem definido

Antes do envio só quem envia mexe na memória; depois, só quem recebe.

A memória continua válida

Exatamente uma task liberta o buffer, e ninguém lhe mexe depois disso.

Nunca dados da stack

Deixam de ser válidos quando o stack frame muda.

```
xPointerQueue = xQueueCreate( 5, sizeof( char * ) );

/* Quem envia */
pcStringToSend = ( char * ) prvGetBuffer( xMaxStringLength );
snprintf( pcStringToSend, xMaxStringLength,
          "String number %d\r\n", xStringNumber );
xQueueSend( xPointerQueue, &pcStringToSend, portMAX_DELAY );

/* Quem recebe */
xQueueReceive( xPointerQueue, &pcReceivedString,
               portMAX_DELAY );
vPrintString( pcReceivedString );
prvReleaseBuffer( pcReceivedString );
```

Nota: envia-se o **endereço do ponteiro** (&pcStringToSend): a queue copia o ponteiro, não a string.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 5

Mailbox: uma queue de comprimento 1

QUEUE

Os dados **passam** pela queue: quem envia coloca, quem recebe retira.

MAILBOX

Os dados **ficam** na mailbox até serem sobrescritos. Qualquer task ou ISR os pode ler, sem os retirar.

Exemplo: a task do sensor escreve a temperatura a cada 100 ms; as outras leem o último valor quando precisam.

Escrever: xQueueOverwrite()

```
xMailbox = xQueueCreate( 1, sizeof( Example_t ) );

xData.ulValue = ulNewValue;
xData.xTimeStamp = xTaskGetTickCount();
xQueueOverwrite( xMailbox, &xData );
```

Só para queues de comprimento 1. Devolve sempre pdPASS.

Ler: xQueuePeek()

```
BaseType_t xQueuePeek( QueueHandle_t xQueue,
                       void * const pvBuffer,
                       TickType_t xTicksToWait );

xQueuePeek( xMailbox, pxData, portMAX_DELAY );
```

Lê da frente **sem remover**. Mesmos parâmetros de xQueueReceive().

MÓDULO 4 EM RESUMO

As queues copiam os dados e bloqueiam quem lê de uma queue vazia ou escreve numa cheia; as prioridades decidem se andam vazias ou cheias.

- xQueueSend / xQueueReceive
- Queue by copy
- Block time
- Estruturas para várias origens
- Ponteiros para dados grandes
- Mailbox

A seguir: sessões práticas 1 e 2, na placa. Depois, o Módulo 5.

FORMAÇÃO · SESSÃO PRÁTICA 1

Ambiente e primeiro projeto

Raspberry Pi Pico 2 nos cores RISC-V, VS Code, pico-sdk, FreeRTOS e um J-Link

A DISTRIBUIÇÃO (LIVRO, CAP. 2)

Ports, ficheiros-fonte, include paths, FreeRTOSConfig.h, convenções

INTEGRAR NO PICO-SDK

Projeto Pico em RISC-V, kernel da Raspberry Pi, configuração

PRIMEIRO PROGRAMA

Duas tasks, uma queue, um LED; hooks e debug com o J-Link

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 2; pico-sdk, pico-examples e raspberrypi/FreeRTOS-Kernel

SESSÃO PRÁTICA 1

No fim desta sessão, cada formando tem

01 **Ferramentas instaladas**
VS Code e extensão Pico a compilar; J-Link a programar a placa.

02 **Kernel descarregado**
O FreeRTOS-Kernel da Raspberry Pi, com o port RP2350_RISC-V.

03 **Projeto base a correr**
Um LED a piscar com o pico-sdk, nos cores RISC-V, ainda sem RTOS.

04 **FreeRTOS integrado**
CMake, FreeRTOSConfig.h e um só core.

05 **Primeiro programa**
Duas tasks e uma queue a controlar o LED.

06 **Rede de segurança**
configASSERT, hooks, RTT e debug com o RTOS a correr.

SESSÃO PRÁTICA 1

O que instalar e ligar antes

Ferramenta	Para quê
VS Code + extensão Raspberry Pi Pico	Instala o pico-sdk, a toolchain RISC-V, CMake, Ninja e picotool; cria e compila projetos. ID: raspberry-pi.raspberry-pi-pico
Raspberry Pi Pico 2 W	RP2350: dois Cortex-M33 e dois Hazard3 RISC-V. Usamos um core RISC-V. LED externo no GP1.
SEGGER J-Link + J-Link Software	Programa a flash e faz debug por SWD. Dispositivo: RP2350_RV32_0. segger.com/downloads/jlink
Cortex-Debug (e RTOS Views)	Extensões do VS Code que ligam o debugger ao J-Link GDB Server. IDs: marus25.cortex-debug · mcu-debug.rtos-views
Cabos e botão	USB para a Pico; SWD do J-Link ao conector de debug; um LED (com resistência) do GP1 ao GND; um botão entre o GP0 e o GND.
FreeRTOS-Kernel da Raspberry Pi + Git	github.com/raspberrypi/FreeRTOS-Kernel, com o port RP2350_RISC-V.

O J-Link precisa de **VTref**: ligar ao 3V3(OUT) da Pico, além de SWCLK, SWDIO e GND. Testar a ligação no J-Link Commander **antes** da sessão.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

Ports e onde obter o código

Port = uma combinação de compilador e processador. O FreeRTOS compila com cerca de 20 compiladores e corre em mais de 40 arquiteturas.

O FreeRTOS é uma **biblioteca de ficheiros C**: compilados com o projeto, tornam a API disponível numa aplicação bare-metal.

Distribuição principal

Kernel, bibliotecas e todas as demos. A melhor forma de começar.

Repositórios individuais

Ex.: só o kernel. Hoje: o fork da Raspberry Pi, que tem o port do RP2350 em RISC-V.

DISTRIBUIÇÃO PRINCIPAL, COM SUBMÓDULOS

```
git clone https://github.com/FreeRTOS/FreeRTOS.git \
    --recurse-submodules
```

```
FreeRTOS
├─Source código-fonte do kernel
└─Demo demos pré-configuradas por port
FreeRTOS-Plus
├─Source outras bibliotecas FreeRTOS
└─Demo demos dessas bibliotecas
```

SÓ O KERNEL, VERSÃO DA RASPBERRY PI

```
git clone https://github.com/raspberrypi/FreeRTOS-Kernel
```

> **Atenção:** o fork está parado desde julho de 2024 (V11.1.0+) e não recebe as correções do kernel oficial.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 2

A DISTRIBUIÇÃO DO FREERTOS

Que ficheiros entram no projeto

Ficheiro	Quando	Nota
tasks.c, list.c	Sempre	Núcleo do kernel
queue.c	Quase sempre	Queues e semáforos
timers.c	Se usar software timers	Módulo 7
event_groups.c	Se usar event groups	
stream_buffer.c	Se usar stream ou message buffers	
croutine.c	Não usar	Co-routines, já sem manutenção
portable/ThirdParty/GCC/RP2350_RISC-V/*	Sempre (o nosso port)	port.c, portASM.S, portmacro.h
portable/MemMang/heap_4.c	Um só heap_n.c	Módulo 2

No repo da Raspberry Pi (**github.com/raspberrypi/FreeRTOS-Kernel**), os ports de terceiros ficam em **portable/ThirdParty/[compilador]/[port]**.

Pico 2 em RISC-V: **RP2350_RISC-V**, da Raspberry Pi.

Com o pico-sdk, o CMake do port escolhe estes ficheiros.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 2

Include paths e headers

TRÊS INCLUDE PATHS

1. Os headers do kernel: **include/**
2. O port: **.../RP2350_RISC-V/include**
3. A pasta com o **FreeRTOSConfig.h**

Com o pico-sdk, os dois primeiros vêm do CMake do port.

EM CADA FICHEIRO QUE USA A API

```
#include "FreeRTOS.h" /* sempre primeiro */
#include "task.h"
#include "queue.h" /* ou semphr.h… */
```

Nunca incluir o FreeRTOSConfig.h diretamente: o FreeRTOS.h inclui-o no momento certo. Nenhum outro header do kernel.

FreeRTOSConfig.h

Adapta o kernel à aplicação: pertence à **aplicação**, não à pasta do código-fonte do FreeRTOS.

Começar pelo da demo do mesmo port. Para o RP2350: **pico-examples/freertos**, FreeRTOSConfig_examples_common.h.

> Valores de partida para esta sessão. O port é SMP: **configNUMBER_OF_CORES = 1** põe-no a correr num core.

Não usados neste port

`configMAX_SYSCALL_INTERRUPT_PRIORITY`
`configKERNEL_INTERRUPT_PRIORITY`
`configCPU_CLOCK_HZ`

Essenciais noutras arquitecturas: ver módulo 5.

```
#define configUSE_PREEMPTION            1
#define configTICK_RATE_HZ              1000
#define configMAX_PRIORITIES            8
#define configMINIMAL_STACK_SIZE        512             /* words */
#define configUSE_IDLE_HOOK             0
#define configUSE_TICK_HOOK             0
#define configTICK_TYPE_WIDTH_IN_BITS   TICK_TYPE_WIDTH_32_BITS
#define configTOTAL_HEAP_SIZE           ( 64 * 1024 )
#define configUSE_MALLOC_FAILED_HOOK    1
#define configCHECK_FOR_STACK_OVERFLOW  2

/* Software timers: usados a partir da sessão 3 */
#define configUSE_TIMERS                1
#define configTIMER_TASK_PRIORITY       ( configMAX_PRIORITIES - 1 )
#define configTIMER_QUEUE_LENGTH        10
#define configTIMER_TASK_STACK_DEPTH    configMINIMAL_STACK_SIZE
#define INCLUDE_xTimerPendFunctionCall  1

/* Funções usadas nos exercícios */
#define INCLUDE_vTaskDelay              1
#define INCLUDE_xTaskDelayUntil         1
#define INCLUDE_vTaskSuspend            1

/* configASSERT() usado no main() */
#include <assert.h>
#define configASSERT( x ) assert( x )
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 2

Tipos de dados e convenções de nomes

TickType_t

Guarda o tick count e especifica tempos. 16, 32 ou 64 bits com configTICK_TYPE_WIDTH_IN_BITS; num RISC-V de 32 bits, 32.

BaseType_t

O tipo mais eficiente da arquitetura (32 bits no Hazard3). Para retornos simples e pdTRUE / pdFALSE.

Prefixos

Variáveis: c char, s int16_t, l int32_t, x BaseType_t e outros, u unsigned, p ponteiro (uc, pc...). Funções: tipo + ficheiro: **vTask**PrioritySet(), **xQueue**Receive(), **pvTimer**GetTimerID(); prv = privada.

PREFIXOS DAS MACROS

Prefixo	Exemplo	Onde está
port	portMAX_DELAY	portable.h / portmacro.h
task	taskENTER_CRITICAL()	task.h
pd	pdTRUE, pdPASS	projdefs.h
config	configUSE_PREEMPTION	FreeRTOSConfig.h
err	errQUEUE_FULL	projdefs.h

pdTRUE = pdPASS = 1; pdFALSE = pdFAIL = 0.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 2

CRIAR UM PROJETO FREERTOS

Dois caminhos; na Pico, um híbrido

COMO COMEÇAR:

Adaptar uma demo (recomendado)

Compilar a demo do port, tirar as demo tasks, deixar no main() só o hardware e o vTaskStartScheduler().

Projeto de raiz

Projeto sem RTOS a funcionar; depois ficheiros do kernel, FreeRTOSConfig.h, include paths, opções de compilação e handlers de interrupção.

O QUE FAZEMOS NA PICO 2

De raiz, com as peças do exemplo oficial

1. **Passo 1:** a extensão Pico cria o projeto, que já compila e corre sem RTOS.
2. **Passo 2:** juntar o kernel (o CMake inclui o FreeRTOS_Kernel_import.cmake da pasta do port) e partir do FreeRTOSConfig.h do **pico-examples/freertos**.
3. **Passo 3:** include paths, opções de compilação e handlers de interrupção ficam a cargo do CMake e do port. Não há nada para fazer à mão.
4. **Passo 4:** escrever as tasks.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 2; documentação Raspberry Pi (pico-sdk)

PASSO 1

Um projeto Pico em RISC-V que já funciona

1. Extensão Pico (barra lateral) → **New C/C++ Project**.
2. Nome e pasta do projeto; placa **Pico 2 W**; em Architecture, ativar **RISC-V**.
3. Escolher a versão do SDK e **Create**: a extensão gera o CMakeLists.txt e um .c com o main().
4. Substituir esse main() pelo **main.c** ao lado.
5. **Compile Project** (botão da extensão).
6. Programar: pelo J-Link ou, sem debugger, arrastar o .uf2 em modo BOOTSEL.

RISC-V = **PICO_PLATFORM rp2350-riscv** no CMake. O RP2350 arranca na arquitetura do binário.

Na Pico 2 W o LED da placa está no chip Wi-Fi. Usamos um **LED externo no GP1**, com resistência de ~330 Ω para o GND.

MAIN.C

```
#include "pico/stdlib.h"
#define LED_PIN 1 /* GP1: LED externo */

int main( void )
{
    gpio_init( LED_PIN );
    gpio_set_dir( LED_PIN, GPIO_OUT );
    for( ;; )
    {
        gpio_xor_mask( 1u << LED_PIN );
        sleep_ms( 500 );
    }
}
```

Se o LED do GP1 pisca, o projeto compila, programa a placa e corre. Só agora entra o FreeRTOS.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 2

Juntar o kernel ao projeto, com CMake

```
Formacao/
├─FreeRTOS-Kernel/ ← git clone
└─MeuProjeto/
  ├─CMakeLists.txt ← editar aqui
  ├─pico_sdk_import.cmake ← da extensão
  ├─FreeRTOSConfig.h
  └─main.c
```

O import usa o FREERTOS_KERNEL_PATH e carrega o port **RP2350_RISC-V**.

FreeRTOS-Kernel-Heap4 traz o kernel, o port e o heap_4; o include path da aplicação dá o FreeRTOSConfig.h.

ACRESCENTAR AO CMAKELISTS.TXT

```
# CMakeLists.txt, depois de pico_sdk_init()
set( FREERTOS_KERNEL_PATH
    ${CMAKE_CURRENT_LIST_DIR}/../FreeRTOS-Kernel )
set( FREERTOS_PORT_PATH
    ${FREERTOS_KERNEL_PATH}/portable/ThirdParty/GCC/RP2350_RISC-V )

include( ${FREERTOS_PORT_PATH}/FreeRTOS_Kernel_import.cmake )

# target_*: só depois do add_executable()
target_include_directories( ${PROJECT_NAME}
    PRIVATE ${CMAKE_CURRENT_LIST_DIR} )
target_link_libraries( ${PROJECT_NAME}
    pico_stdlib
    FreeRTOS-Kernel-Heap4 )
```

- set KERNEL_PATH: onde está o kernel (um nível acima)
- set PORT_PATH: a pasta do port RP2350_RISC-V
- include: corre o import: cria as bibliotecas FreeRTOS-Kernel-*
- target_include_directories: a pasta do FreeRTOSConfig.h
- target_link_libraries: junta ao build o pico-sdk, o kernel, o port e o heap_4

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 3

O que o port faz por nós: tick e interrupções

```
/* port.c do RP2350_RISC-V (excerto) */
void vPortSetupTimerInterrupt( void )
{
    riscv_timer_set_fullspeed( true );
    uxTimerIncrementsForOneTick =
        clock_get_hz( clk_sys ) / configTICK_RATE_HZ;
    /* … programa o mtimecmp do SIO … */
    irq_set_exclusive_handler( SIO_IRQ_MTIMECMP,
        freertos_risc_v_mtimer_interrupt_handler );
    irq_set_enabled( SIO_IRQ_MTIMECMP, true );
}
```

Quem chama isto?

```
main() → vTaskStartScheduler()
  → xPortStartScheduler() /* port */
    → vPortSetupTimerInterrupt()
```

Nós arrancamos; o port prepara o tick e os handlers.

O tick

Vem do machine timer (mtime / mtimecmp) do **SIO**, o bloco de I/O de ciclo único do RP2350, e não de um SysTick.

Os handlers (no Cortex-M, à mão)

Tick (≈ SysTick): mtimer handler
Troca de task (≈ PendSV/SVC): trap handler, via ecall
IRQs do pico-sdk: interrupt handler, para trocar de task à saída

As interrupções

Secções críticas desligam todas. Sem nesting e sem configMAX_SYSCALL_INTERRUPT_PRIORITY (módulo 5).

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 4

O primeiro programa: duas tasks e uma queue

PRIORIDADE 1: ENVIA A CADA 500 MS

```
static QueueHandle_t xLedQueue;

static void vSenderTask( void *pvParameters )
{
    TickType_t xLastWake = xTaskGetTickCount();
    uint32_t ulCount = 0;
    for( ;; )
    {
        vTaskDelayUntil( &xLastWake,
                         pdMS_TO_TICKS( 500 ) );
        ulCount++;
        xQueueSend( xLedQueue, &ulCount, 0 );
    }
}
```

PRIORIDADE 2: TROCA O LED

```
static void vLedTask( void *pvParameters )
{
    uint32_t ulReceived;
    for( ;; )
    {
        /* portMAX_DELAY: só retorna com dados */
        xQueueReceive( xLedQueue, &ulReceived,
                       portMAX_DELAY );
        gpio_xor_mask( 1u << LED_PIN );
    }
}
```

O recetor tem mais prioridade, por isso a queue nunca tem mais de um item.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 4

O main(): criar e arrancar

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

int main( void )
{
    stdio_init_all();
    gpio_init( LED_PIN );
    gpio_set_dir( LED_PIN, GPIO_OUT );
    xLedQueue = xQueueCreate( 4, sizeof( uint32_t ) );
    configASSERT( xLedQueue ≠ NULL );
    xTaskCreate( vLedTask, "LED", 256, NULL, 2, NULL );
    xTaskCreate( vSenderTask, "Sender", 256, NULL, 1,
                 NULL );
    vTaskStartScheduler();
    for( ;; ); /* só chega aqui se faltar heap */
}
```

A estrutura

Preparar o hardware, criar tasks e objetos, arrancar o scheduler. Aqui o hardware é preparado com o pico-sdk.

256 words = 1 KB por task

Generoso para começar. Afinar depois com o high water mark.

Sai o ciclo do passo 1

O for(;;) com sleep_ms() deixa de ser preciso: o scheduler nunca devolve o controlo.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 5

Hooks e mensagens pelo J-Link (RTT)

Obrigatórios com a configuração do passo 2

```
void vApplicationStackOverflowHook( TaskHandle_t xTask,
                                    char *pcTaskName )
{
    ( void ) xTask;
    ( void ) pcTaskName; /* ver no debugger */
    taskDISABLE_INTERRUPTS();
    for( ;; );
}

void vApplicationMallocFailedHook( void )
{
    taskDISABLE_INTERRUPTS();
    for( ;; );
}
```

PRINTF PELO J-LINK: SEGGER RTT

```
# CMakeLists.txt, depois do add_executable()
pico_enable_stdio_rtt( ${PROJECT_NAME} 1 )
pico_enable_stdio_usb( ${PROJECT_NAME} 0 )
pico_enable_stdio_uart( ${PROJECT_NAME} 0 )
```

Ver as mensagens

J-Link RTT Viewer (ou a consola RTT do Cortex-Debug), com o dispositivo RP2350_RV32_0.

Porquê RTT

Não precisa de UART nem de USB: o texto passa por um buffer na RAM que o J-Link lê.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

PASSO 5

Programar e fazer debug com o J-Link

.vscode/launch.json: uma configuração nova

```
{
  "name": "Debug (J-Link)",
  "cwd": "${workspaceRoot}",
  "executable": "${command:raspberry-pi-pico.launchTargetPath}",
  "request": "launch", "type": "cortex-debug",
  "servertype": "jlink",
  "serverpath": "C:/Program Files/SEGGER/JLink_V986a/JLinkGDBServerCL.exe",
  "gdbPath": "${command:raspberry-pi-pico.getGDBPath}",
  "device": "RP2350_RV32_0", "interface": "swd",
  "rtos": "FreeRTOS", "serialNumber": "",
"svdFile": "${userHome}/.pico-sdk/sdk/2.3.1/src/${command:raspberry-pi-
pico.getChip}/hardware_regs/${command:raspberry-pi-
pico.getChipUppercase}.svd",  "runToEntryPoint": "main",
  "overrideLaunchCommands": [
    "monitor reset",
    "load \"${command:raspberry-pi-pico.launchTargetPath}\""
  ],
  "rttConfig": {
    "enabled": true, "address": "auto",
    "decoders": [ { "port": 0, "type": "console", "label": "RTT" } ]
  }
}
```

Compilar e programar

F5 com esta configuração: o J-Link grava a flash e pára no main(). O LED deve piscar a 1 Hz.

Breakpoints nas tasks

Parar em vLedTask, ver a stack de chamadas, mudar o período e confirmar.

Vista das tasks

Extensão RTOS Views: tasks, estados e stacks do FreeRTOS. Confirmar em RISC-V.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

SESSÃO PRÁTICA 1

Erros típicos no primeiro build

Sintoma	Causa provável
FreeRTOS location was not specified	Falta a pasta FreeRTOS-Kernel no projeto, ou FREERTOS_KERNEL_PATH.
FreeRTOSConfig.h: No such file	Falta target_include_directories com a pasta do FreeRTOSConfig.h.
O build usa o port RP2350_ARM_NTZ	PICO_PLATFORM não é rp2350-riscv: escolher RISC-V na extensão.
undefined reference to vApplication...Hook	Falta um hook ativado no FreeRTOSConfig.h (passo 5).
vTaskStartScheduler() retorna	Heap insuficiente para a idle task: subir configTOTAL_HEAP_SIZE.
J-Link: Could not connect to target	VTref sem 3V3, SWD mal ligado, ou dispositivo errado (RP2350_RV32_0).
O assert não pára nada	Build Release: o NDEBUG desliga o assert() do configASSERT.
O RTT Viewer não mostra nada	Falta pico_enable_stdio_rtt, ou o viewer não encontrou o bloco RTT.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

SESSÃO PRÁTICA 1

Exercícios sobre o projeto base

AGORA · MÓDULOS 1 A 4

01 Botão hardware extra: botão

Uma terceira task lê o botão no GP0 e envia um novo período ao sender por outra queue.

02 Heap

Ver xPortGetFreeHeapSize() e xPortGetMinimumEverFreeHeapSize() antes e depois de criar as tasks.

03 Prioridades

Pôr as duas tasks na mesma prioridade e depois inverter. O que muda na queue?

TRABALHO FUTURO

04 ISR hardware extra: botão módulo 5

Interrupção no GP0 com gpio_set_irq_enabled_with_callback() e xQueueSendFromISR().

05 Software timer módulo 7

Com configUSE_TIMERS = 1, trocar a sender task por um timer auto-reload.

06 Stack módulo 8

Ver uxTaskGetStackHighWaterMark() de cada task (RTT ou debugger) e reduzir as stacks.

Fonte: documentação Raspberry Pi (pico-sdk, FreeRTOS-Kernel) e SEGGER (J-Link)

SESSÃO PRÁTICA 1 EM RESUMO

Primeiro um projeto que corre sem RTOS; depois o kernel, um passo de cada vez.

Port = compilador + CPU

RP2350_RISC-V

FreeRTOS_Kernel_import.cmake

FreeRTOS-Kernel-Heap4

FreeRTOSConfig.h na aplicação

configNUMBER_OF_CORES = 1

Tick = mtime

J-Link + RTT

Hooks

Este projeto é a base das outras sessões práticas.

FORMAÇÃO · SESSÃO PRÁTICA 2 EXTRA

Preempção e prioridades

Os exemplos do capítulo 4, numa placa real

VER O SCHEDULER

Contadores no debugger e um pino por task, com trace hooks

OS EXEMPLOS OFICIAIS

Time slicing, starvation, bloqueio, preempção, mudar prioridades

MEXER NA CONFIGURAÇÃO

Sem time slicing e em modo cooperativo

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 4, e exemplos do repositório FreeRTOS-Kernel-Book

SESSÃO PRÁTICA 2

Os exemplos oficiais que vamos usar

Livro: *Mastering the FreeRTOS Real Time Kernel* (V1.1.0) · github.com/FreeRTOS/FreeRTOS-Kernel-Book

Exemplo do livro	O que faz	O que mostra	Exercício
4.1	Criar duas tasks, mesma prioridade	Time slicing	1
4.2	Parâmetro da task	Uma função, duas instâncias	Código base
4.3	Prioridades diferentes, null loop	Starvation	2
4.4	vTaskDelay()	Blocked; a idle task aparece	3
4.5	vTaskDelayUntil()	Período fixo	3
4.6	Contínuas + periódica	Preempção	4
4.7	Idle hook	Medir o CPU livre	3
4.8	vTaskPrioritySet()	Prioridades em run time	5
4.9	vTaskDelete()	A idle task liberta a memória	Leitura
Secção 4.12	Sem time slicing; cooperativo	Figuras 4.21 e 4.22	6

Os exemplos do livro

Example001 a 026 (001 = 4.1 … 009 = 4.9). Correm no simulador Windows e escrevem com printf.

github.com/FreeRTOS/FreeRTOS-Kernel-Book/tree/main/examples/Win32-simulator-MSVC

O essencial

O mínimo para criar tasks, escolher prioridades e perceber quando cada uma corre.

Hoje: exemplos 4.1 a 4.8 e secção 4.12, na Pico 2. O 4.9 (vTaskDelete) fica como leitura.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 4 (exemplos 4.1–4.9 e secção 4.12)

SESSÃO PRÁTICA 2

Como ver o scheduler a trabalhar

SEM HARDWARE EXTRA

Contadores no debugger

Cada task incrementa o seu contador. Com as live expressions / watch do debugger vê-se quem corre e quanto. Uma task em starvation fica em 0.

O MELHOR PARA VER PREEMPÇÃO

Um pino por task

Os trace hooks põem o pino da task a 1 quando entra em Running e a 0 quando sai. Com um analisador lógico barato vê-se a timeline real, ao microssegundo.

RESUMO EM NÚMEROS

Run-time stats

vTaskGetRunTimeStatistics() dá a percentagem de CPU de cada task (módulo 8). Bom para confirmar starvation e CPU livre.

Evitar printf() para isto: é lento, muda o próprio timing que queremos ver e pode nem ser thread-safe (módulo 8).

SESSÃO PRÁTICA 2

Um pino por task, com trace hooks

```
/* FreeRTOSConfig.h */
#define configUSE_APPLICATION_TASK_TAG 1
#define INCLUDE_xTaskGetIdleTaskHandle 1
void vTraceIn( void *pvTag );
void vTraceOut( void *pvTag );
/* pxCurrentTCB só existe dentro de tasks.c, onde os macros são expandidos */
#define traceTASK_SWITCHED_IN() vTraceIn( ( void * ) pxCurrentTCB->pxTaskTag )
#define traceTASK_SWITCHED_OUT() vTraceOut( ( void * ) pxCurrentTCB->pxTaskTag )
```

```
/* main.c: a tag de cada task é a máscara do seu pino (registos SIO) */
void vTraceIn( void *pvTag ) { if( pvTag != NULL ) sio_hw->gpio_set = ( uint32_t ) pvTag; }
void vTraceOut( void *pvTag ) { if( pvTag != NULL ) sio_hw->gpio_clr = ( uint32_t ) pvTag; }
/* no início de cada task */
vTaskSetApplicationTaskTag( NULL, ( TaskHookFunction_t ) ( 1u << 2 ) ); /* GP2 */
/* idle: a partir de outra task, já com o scheduler a correr */
vTaskSetApplicationTaskTag( xTaskGetIdleTaskHandle(), ( TaskHookFunction_t ) ( 1u << 6 ) ); /* GP6 */
```

Pinos livres como saídas: ex. GP2 a GP5, e GP6 para a idle.

Evitar o GP1 (LED) e o GP0 (botão das sessões 3 e 4).

A idle não corre código nosso: a tag dela é posta por outra task.

Fonte: Livro, cap. 4 e cap. 12 (trace hook macros), adaptados à Pico 2

CÓDIGO BASE · EXEMPLOS 4.1 E 4.2

A task de trabalho contínuo

```
typedef struct
{
    volatile uint32_t ulCount; /* ver no debugger */
    uint32_t ulPin; /* máscara do pino */
} TaskInfo_t;
static TaskInfo_t xT1 = { 0, 1u << 2 }; /* GP2 */
static TaskInfo_t xT2 = { 0, 1u << 3 }; /* GP3 */
static void vContinuousTask( void *pvParameters )
{
    TaskInfo_t *pxInfo = ( TaskInfo_t * ) pvParameters;
    vTaskSetApplicationTaskTag( NULL,
            ( TaskHookFunction_t ) pxInfo->ulPin );
    for( ;; )
    {
        pxInfo->ulCount++; /* trabalho contínuo */
    }
}
```

Uma task que nunca bloqueia: em vez de imprimir e esperar num null loop, só conta.

Uma função, várias instâncias: o parâmetro diz a cada uma o seu contador e o seu pino.

No main()

```
xTaskCreate( vContinuousTask, "T1",
             256, &xT1, 1, NULL );
xTaskCreate( vContinuousTask, "T2",
             256, &xT2, 1, NULL );
vTaskStartScheduler();
```

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 1 · EXEMPLO 4.1

Duas tasks, a mesma prioridade

FAZER

1. T1 e T2 com prioridade 1, como no main() do slide anterior.
2. Correr alguns segundos e parar.
3. Ver os contadores e os pinos.

PREVER ANTES DE CORRER

Quem corre? As duas partilham o CPU de forma igual?

OBSERVAR

Os pinos alternam a cada tick (1 ms com configTICK_RATE_HZ = 1000): é o time slicing. Os contadores ficam parecidos, mas não iguais: round robin não garante tempo igual. O pino da idle nunca sobe.

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 2 · EXEMPLO 4.3

Prioridades diferentes: starvation

FAZER

1. Mudar T2 para prioridade 2.
2. Correr e comparar os contadores.
3. Olhar para o pino de T1.

PREVER ANTES DE CORRER

T1 alguma vez corre?

OBSERVAR

O contador de T1 fica em 0 e o seu pino nunca sobe. T2 está sempre pronta e é a mais prioritária: T1 sofre starvation. Nem a idle task corre.

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 3 · EXEMPLOS 4.4 E 4.7

Bloquear em vez de ocupar o CPU

FAZER

1. Manter T1 = 1 e T2 = 2, mas com vTaskDelay() no loop.
2. Ativar o idle hook (configUSE_IDLE_HOOK = 1) com um contador.
3. Comparar os três contadores.

PREVER ANTES DE CORRER

T1 volta a correr? Quanto tempo sobra para a idle task?

OBSERVAR

As duas correm, cada uma durante microssegundos a cada 10 ms. O pino da idle está alto quase sempre: o contador da idle mede o CPU livre.

```
for( ;; )
{
    pxInfo→ulCount++;
    vTaskDelay( pdMS_TO_TICKS( 10 ) );
}

void vApplicationIdleHook( void )
{
    ulIdleCycleCount++;
}
```

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 4 · EXEMPLO 4.6

Contínuas e periódica: preempção

FAZER

1. T1 e T2 contínuas, prioridade 1 (como no exercício 1).
2. T3 periódica, prioridade 2, a cada 3 ms.
3. Medir no analisador lógico.

PREVER ANTES DE CORRER

O que acontece a T1 e T2 de 3 em 3 ms?

OBSERVAR

T1 e T2 alternam a cada tick; de 3 em 3 ms T3 preempta a que estiver a correr, faz o seu trabalho curto e bloqueia.

```
static void vPeriodicTask( void *pvParameters )
{
    TickType_t xLastWake = xTaskGetTickCount();
    for( ;; )
    {
        ulPeriodicCount++;
        vTaskDelayUntil( &xLastWake,
                         pdMS_TO_TICKS( 3 ) );
    }
}
```

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 5 · EXEMPLO 4.8

Mudar prioridades em run time

FAZER

1. T1 começa com prioridade 2 e T2 com 1 (guardar o handle de T2).
2. T1 sobe T2 acima de si; T2 baixa-se abaixo de T1.
3. Ver a sequência nos pinos.

PREVER ANTES DE CORRER

Alguma delas bloqueia? Como é que alternam?

OBSERVAR

Nenhuma bloqueia, mas alternam a cada 200 ms: cada vTaskPrioritySet() provoca uma troca imediata, sem esperar pelo tick.

```
/* FreeRTOSConfig.h */
#define INCLUDE_vTaskPrioritySet  1
#define INCLUDE_uxTaskPriorityGet 1

for( ;; ) /* T1, prioridade 2 */
{
    ulT1Count++;
    busy_wait_ms( 200 ); /* ocupada, sem bloquear */
    vTaskPrioritySet( xT2Handle,
        uxTaskPriorityGet( NULL ) + 1 );
}

for( ;; ) /* T2, prioridade 1 */
{
    ulT2Count++;
    busy_wait_ms( 200 ); /* ocupada, sem bloquear */
    vTaskPrioritySet( NULL,
        uxTaskPriorityGet( NULL ) - 2 );
}
```

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

EXERCÍCIO 6 · SECÇÃO 4.12

Mudar o algoritmo de scheduling

Repetir os exercícios 1 e 4 com outra configuração. **Escrever a previsão antes de correr.**

configUSE_...	Sobre	O que se deve ver
TIME_SLICING = 0	Ex. 1	Quem entra em Running fica até bloquear ou ser preemptada: uma das duas pode nunca largar o CPU.
TIME_SLICING = 0	Ex. 4	T3 continua a preemptar; entre pulsos, T1 e T2 já não alternam a cada tick.
PREEMPTION = 0	Ex. 4	T3 fica Ready de 3 em 3 ms, mas só corre quando a task em Running bloqueia ou cede. Com contínuas, nunca.
PREEMPTION = 0 + taskYIELD()	Ex. 4	Com taskYIELD() no loop de T1 e T2, T3 volta a correr, mas com atraso. Medir o atraso.

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

SESSÃO PRÁTICA 2

Perguntas para fechar

Porque é que uma task contínua só faz sentido na prioridade mais baixa?	Ex. 2
Porque é que T1 e T2 não recebem exatamente o mesmo tempo com time slicing?	Ex. 1
O que mede o contador do idle hook, e porque interessa num produto?	Ex. 3
A preempção só acontece no tick?	Ex. 4, 5
Que custo tem desligar o time slicing? E a preempção?	Ex. 6

Fonte: Livro, cap. 4 e exemplos Win32 001–009, adaptados à Pico 2

SESSÃO PRÁTICA 2 EM RESUMO

Corre sempre a task pronta de maior prioridade. Tudo o resto decorre disto e de quando as tasks bloqueiam.

- Time slicing
- Starvation
- Blocked liberta o CPU
- Idle = CPU livre
- Preempção fora do tick
- configUSE_PREEMPTION
- configUSE_TIME_SLICING

Guardar o projeto com os exemplos: é a base da sessão prática 3.

PLANO DA FORMAÇÃO · DIA 2

Módulos e sessões práticas, pela ordem da formação

01	Módulo 1 · Fundamentos de sistemas operativos de tempo real	manhã	dia 1
02	Módulo 2 · Gestão de memória	manhã	dia 1
03	Módulo 3 · Gestão de tasks	manhã	dia 1
04	Módulo 4 · Filas (queues)	manhã	dia 1
P1	Prática 1 · Ambiente e primeiro projeto	tarde	dia 1
P2	Prática 2 · Preempção e prioridades EXTRA	tarde	dia 1
05	**Módulo 5 · Gestão de interrupções**	manhã	dia 2
06	**Módulo 6 · Gestão de recursos**	manhã	dia 2
07	**Módulo 7 · Software timers**	manhã	dia 2
08	**Módulo 8 · Troubleshooting e debug**	manhã	dia 2
P3	**Prática 3 · Interrupções, recursos, timers e debug**	tarde	dia 2
P4	**Prática 4 · Aprofundar: ISRs, recursos, timers e debug** EXTRA	tarde	dia 2

FORMAÇÃO · MÓDULO 5

Gestão de interrupções

Sincronizar interrupções com tasks

A API NAS ISRs

Funções FromISR, xHigherPriorityTaskWoken, portYIELD_FROM_ISR

DEFERRED PROCESSING

Semáforos binários e de contagem, daemon task, queues

PRIORIDADES

Nesting, configMAX_SYSCALL_INTERRUPT_PRIORITY, notas Cortex-M e RISC-V

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 7

GESTÃO DE INTERRUPÇÕES

Eventos, ISRs e tasks

Um sistema embebido reage a eventos do ambiente: um pacote chega à Ethernet (evento) e tem de ir para a stack TCP/IP (ação). Para cada evento há três decisões:

01

Como detetar?

Normalmente com interrupções; também se pode fazer polling.

02

Quanto fazer na ISR?

Em geral, a ISR deve ser o mais curta possível.

03

Como passar à task?

E como estruturar o código para eventos assíncronos.

⚠ Prioridade de task ≠ prioridade de interrupção. As tasks são software; a ISR é decidida pelo hardware. **A interrupção de prioridade mais baixa interrompe a task de prioridade mais alta**, e nenhuma task preempta uma ISR.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

GESTÃO DE INTERRUPÇÕES

A API ...FromISR()

Muitas funções da API fazem coisas inválidas numa ISR. A mais óbvia: pôr a task que chamou em **Blocked**. Numa ISR não há task que chamou.

Por isso algumas funções existem em duas versões: uma para tasks e outra, terminada em **FromISR**, para ISRs.

Nunca chamar numa ISR uma função da API sem "FromISR" no nome.

VANTAGEM

Código de task e de ISR mais eficiente, e entrada nas interrupções mais simples. Sem ela, cada função teria de descobrir o contexto em que corre.

DESVANTAGEM

O código tem de saber em que contexto corre. Um driver de terceiros chamado da ISR e de uma task não sabe qual das versões usar. Soluções:

- Adiar o processamento para uma task
- Com nesting, usar sempre a FromISR (não na Pico 2)
- Camada de abstração que testa o contexto

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

GESTÃO DE INTERRUPÇÕES

xHigherPriorityTaskWoken e portYIELD_FROM_ISR()

```
void vExampleISR( void )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    uint32_t ulValor = LER_REGISTO_DO_PERIFERICO();

    /* Pode pôr em Ready uma task mais prioritária */
    xQueueSendToBackFromISR( xQueue, &ulValor,
                             &xHigherPriorityTaskWoken );

    /* Se sim, pedir a troca de contexto à saída */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Protótipo da ISR e macro de yield dependem do port: ver a documentação do port usado.

Chamada numa task

Com preempção, a troca para a task mais prioritária acontece **dentro** da função da API.

Chamada numa ISR

A troca **não** é automática. A função põe *pxHigherPriorityTaskWoken a pdTRUE; a ISR decide pedir a troca. Se não pedir, no pior caso espera pelo próximo tick.

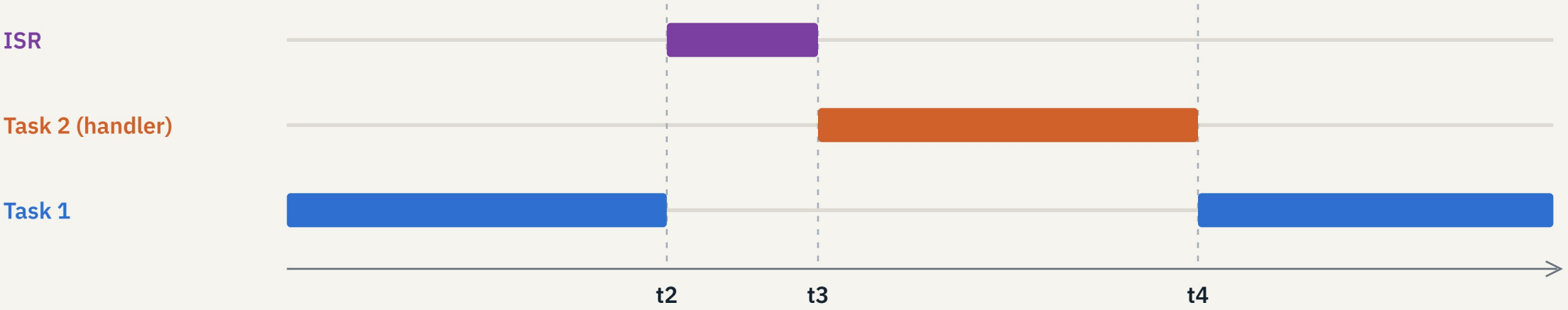
Inicializar a pdFALSE; a mesma variável serve para várias chamadas. Pode ser NULL se não for usada.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7; exemplo adaptado

GESTÃO DE INTERRUPÇÕES

Deferred interrupt processing

A ISR regista a causa e limpa a interrupção; o resto é **adiado** para uma task. Se essa task tiver a prioridade mais alta, o processamento acontece logo a seguir, como se fosse na ISR.



Porquê ISRs curtas: tasks só correm sem ISRs ativas; ISRs criam jitter; podem bloquear outras interrupções; nesting fica mais provável.

Quando adiar: processamento não trivial (ex.: filtrar uma leitura ADC), ações proibidas numa ISR (consola, alocar memória), duração imprevisível.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

Sincronizar uma task com uma interrupção

Um **semáforo binário** é, conceptualmente, uma queue de comprimento 1: está sempre vazio ou cheio. A task bloqueia a tentar tirá-lo; a ISR dá-o e desbloqueia a task.

Aqui a ISR dá sem ter tirado, e a task tira sem devolver: **não é** o uso do semáforo para proteger recursos (módulo seguinte).

As task notifications fazem o mesmo de forma mais eficiente.

Exemplo: a task e a ISR

xSemaphoreCreateBinary()	Devolve SemaphoreHandle_t, ou NULL sem heap. Há versão Static.
xSemaphoreTake(xSem, xTicksToWait)	Nunca numa ISR. pdPASS se obteve; pdFALSE se o tempo expirou.
xSemaphoreGiveFromISR(xSem, &woken)	pdFAIL se o semáforo já estava disponível.

HANDLER TASK, PRIORIDADE 3

```
static void vHandlerTask( void *pvParameters )
{
    for( ;; )
    {
        xSemaphoreTake( xBinarySemaphore,
                        portMAX_DELAY );
        /* processar o evento */
    }
}
```

ISR: DÁ O SEMÁFORO

```
void vExampleISR( void )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /* Desbloqueia a handler task */
    xSemaphoreGiveFromISR( xBinarySemaphore,
                           &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

GESTÃO DE INTERRUPÇÕES

Uma handler task robusta

```
static void vUARTReceiveHandlerTask( void *pvParameters )
{
    const TickType_t xMaxExpectedBlockTime =
        pdMS_TO_TICKS( 500 );
    for( ;; )
    {
        if( xSemaphoreTake( xBinarySemaphore,
                xMaxExpectedBlockTime ) == pdPASS )
        {
            /* Processar TODOS os eventos pendentes */
            while( UART_RxCount() > 0 )
                UART_ProcessNextRxEvent();
        }
        else /* nada no tempo esperado */
            UART_ClearErrors();
    }
}
```

Processar tudo o que está pendente

Um semáforo binário só guarda um evento. Se chegarem mais interrupções antes de a task acabar, perdem-se: à terceira, o semáforo já está dado.

Usar timeout, não portMAX_DELAY

Se um erro de hardware impedir a interrupção, a task deteta-o e trata-o em vez de esperar para sempre.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

Semáforos de contagem

Uma queue de comprimento maior que 1, em que só interessa **quantos** itens tem: a contagem. Requer **configUSE_COUNTING_SEMAPHORES = 1**.

USO 1

Contar eventos

O handler dá a cada evento (+1); a task tira a cada evento processado (−1). A contagem é a diferença entre ocorridos e processados.

Contagem inicial: **0**.

USO 2

Gerir recursos

A contagem é o número de recursos livres. Tirar para usar, dar para devolver; em 0 não há recursos.

Contagem inicial: **igual ao máximo**.

```
SemaphoreHandle_t xSemaphoreCreateCounting(
    UBaseType_t uxMaxCount, UBaseType_t uxInitialCount );
xCountingSemaphore = xSemaphoreCreateCounting( 10, 0 );
```

Exemplo anterior: máximo 10, começa em 0. A ISR dá o semáforo 3 vezes; a task processa os 3 eventos, sem perder nenhum.

Adiar para a daemon task

Em vez de uma task por interrupção, a ISR manda a **daemon task** do RTOS executar uma função: **centralized deferred interrupt processing**.

```
/* Função adiada: protótipo PendedFunction_t */
static void vDeferredHandlingFunction(
        void *pvParameter1, uint32_t ulParameter2 );

/* Na ISR */
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
xTimerPendFunctionCallFromISR( vDeferredHandlingFunction,
                               NULL,
                               ulParameterValue,
                               &xHigherPriorityTaskWoken );
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
```

Usa a timer command queue. Devolve pdFAIL se essa queue estiver cheia.

VANTAGENS

- Menos recursos: sem task por interrupção
- A função adiada é uma função C normal

DESVANTAGENS

- Tudo corre à prioridade da daemon task (configTIMER_TASK_PRIORITY)
- Menos determinismo: comandos já na queue são tratados primeiro

É comum usar os dois métodos na mesma aplicação.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

Queues dentro de uma ISR

Semáforos comunicam **eventos**. Queues comunicam eventos **e dados**.

```
BaseType_t xQueueSendToBackFromISR(
    QueueHandle_t xQueue,
    const void *pvItemToQueue,
    BaseType_t *pxHigherPriorityTaskWoken );
/* também: xQueueSendToFrontFromISR(),
   xQueueSendFromISR(), xQueueReceiveFromISR() */
```

Retorno: pdPASS, ou errQUEUE_FULL se a queue estiver cheia. Sem block time: numa ISR não se bloqueia.

> Uma queue é **ineficiente** se os dados chegarem a alta frequência. Os drivers de UART das demos usam-na para demonstrar e carregar o sistema, não como exemplo de produção.

ALTERNATIVAS PARA PRODUÇÃO

- DMA recebe os dados; a ISR de fim de transferência dá um semáforo à task
- Copiar para um buffer em RAM e dar um semáforo no fim da mensagem
- Processar na ISR e enviar só o resultado pela queue

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

Interrupt nesting

Na Pico 2: desligado no port RP2350_RISC-V

configMAX_SYSCALL_INTERRUPT_PRIORITY

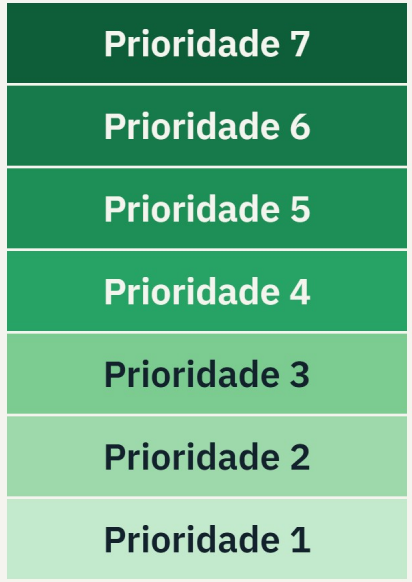
(ou configMAX_API_CALL_INTERRUPT_PRIORITY, nos ports mais recentes) Prioridade mais alta a partir da qual se pode chamar a API FromISR.

configKERNEL_INTERRUPT_PRIORITY

Prioridade do tick interrupt. Tem de ser sempre a mais baixa possível.

Prioridade **numérica** (o número) ≠ prioridade **lógica** (quem tem precedência). A relação depende da arquitetura.

Exemplo: 7 prioridades, 7 = logicamente mais alta, KERNEL = 1, MAX_SYSCALL = 3.



4 a 7: nunca atrasadas pelo kernel, podem fazer nesting, mas **não podem usar a API**. Ex.: controlo de motores.

1 a 3: podem usar a API FromISR e fazer nesting, mas são mascaradas pelas critical sections.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 7

Nota para RISC-V: o Hazard3 no RP2350

	Cortex-M (ex.: ARM_CM4F)	Port RP2350_RISC-V (Pico 2)
Tick	SysTick	Machine timer do SIO (mtime / mtimecmp)
Handlers do kernel	#define para os nomes do vetor	Instalados pelo port em run time (API de IRQs do pico-sdk)
Secção crítica	BASEPRI: só mascara até configMAX_SYSCALL	Desliga todas as interrupções
Nesting de ISRs	Sim, com prioridades	Não: o port desliga a preempção de IRQs
Que ISRs podem usar FromISR	Só as de prioridade lógica ≤ configMAX_SYSCALL	Todas
Verificação com configASSERT	vPortValidateInterruptPriority	Não há

Mais simples: não há prioridades de interrupção a acertar com o kernel.

Mais exigente: sem nesting, uma ISR longa atrasa todas as outras. ISRs curtas e deferred processing.

Fonte: port RP2350_RISC-V (raspberrypi/FreeRTOS-Kernel)

MÓDULO 5 EM RESUMO

A ISR regista o evento e sai depressa; uma task, acordada por um semáforo ou por uma queue, faz o resto.

...FromISR()
portYIELD_FROM_ISR
Deferred processing
Semáforo binário e de contagem
xTimerPendFunctionCallFromISR
Queues em ISRs
configMAX_SYSCALL_INTERRUPT_PRIORITY

A seguir, no Módulo 6: gestão de recursos, com mutexes e semáforos.

FORMAÇÃO · MÓDULO 6

Gestão de recursos

Critical sections, mutexes e gatekeeper tasks

O PROBLEMA

Acesso concorrente, operações não atómicas, reentrância

AS FERRAMENTAS

Critical sections, suspender o scheduler, mutexes, gatekeeper tasks

OS RISCOS

Priority inversion e deadlock

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 8

GESTÃO DE RECURSOS

O problema: acessos interrompidos a meio

Se uma task sai de Running a meio de usar um recurso, este pode ficar inconsistente, e outra task ou ISR que lhe aceda corrompe os dados.

PERIFÉRICOS

Duas tasks a escrever no LCD

```
Hello wAbort, Retry, Fail?orld
```

Task A escreve “Hello world”, é preemptada; **Task B** escreve a sua mensagem; A acaba “orld”.

READ, MODIFY, WRITE

Uma linha de C, quatro instruções

C:

```
PORTA ⊨ 0x01;
```

O compilador gera (pseudo-assembly):

```
LOAD R1,[#PORTA] ; ler PORTA para R1
MOVE R2,#0x01 ; constante 1 para R2
OR R1,R2 ; R1 = R1 | R2
STORE R1,[#PORTA] ; escrever em PORTA
```

Se B alterar PORTA entre o LOAD e o STORE de A, A escreve um valor desatualizado e apaga a alteração de B.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

Funções reentrantes

Uma função é **reentrante** (thread-safe) se pode ser chamada por várias tasks, ou por tasks e ISRs, sem risco. Cada task tem a sua stack e os seus registos: se a função só usa dados da stack ou dos registos, é reentrante.

REENTRANTE

```
long lAddOneHundred( long lVar1 )
{
    long lVar2; /* stack ou registo */
    lVar2 = lVar1 + 100;
    return lVar2;
}
```

NÃO REENTRANTE

```
long lVar1; /* global */
long lNonsenseFunction( void )
{
    static long lState = 0; /* partilhada */
    ...
}
```

A libc também tem estado global (ex.: **errno**, **strtok()**). Duas tasks a usá-las têm o mesmo problema.

GESTÃO DE RECURSOS

Exclusão mútua

Objetivo: quando uma task começa a usar um recurso partilhado não reentrante, tem acesso **exclusivo** até o deixar de novo num estado consistente.

O melhor método: desenhar a aplicação para que os recursos **não sejam partilhados**, e cada um seja usado por uma só task. Nem sempre é prático.

AS FERRAMENTAS DO FREERTOS, DESTE MÓDULO

01
Critical sections
Desligar interrupções durante um troço muito curto.

02
Suspender o scheduler
Impedir trocas de task, com interrupções ligadas.

03
Mutexes
Um token associado ao recurso: tirar, usar, devolver.

04
Gatekeeper tasks
Uma só task é dona do recurso; as outras pedem-lhe.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

GESTÃO DE RECURSOS

Critical sections

NUMA TASK

```
taskENTER_CRITICAL();

/* Nenhuma troca de task aqui. Só correm ISRs
   acima de configMAX_SYSCALL_INTERRUPT_PRIORITY,
   que não podem usar a API. */
PORTA ⊨ 0x01;

taskEXIT_CRITICAL();
```

Numa ISR (ports com nesting)

```
UBaseType_t uxSavedInterruptStatus;
uxSavedInterruptStatus = taskENTER_CRITICAL_FROM_ISR();
/* ... */
taskEXIT_CRITICAL_FROM_ISR( uxSavedInterruptStatus );
```

Como funcionam

Desligam as interrupções, todas ou até configMAX_SYSCALL_INTERRUPT_PRIORITY. Sem interrupções não há preempção.

Muito curtas

Senão atrasam a resposta às interrupções. Rápidas a entrar e a sair, e determinísticas. Não servem para stdout.

Podem ser aninhadas

O kernel conta a profundidade. É a única forma legítima de uma task mudar o estado das interrupções.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

Suspender o scheduler

Também cria uma critical section, mas sem desligar interrupções: impede as trocas de contexto. Um pedido de troca feito por uma ISR fica pendente até ao fim.

```
void vPrintString( const char *pcString )
{
    vTaskSuspendAll();
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    xTaskResumeAll();
}
```

xTaskResumeAll() devolve pdTRUE se fez uma troca pendente. Também pode ser aninhado.

	Critical section	Scheduler suspenso
Protege de	Tasks e interrupções	Só de outras tasks
Interrupções	Desligadas	Continuam ligadas
Duração	Muito curta	Pode ser mais longa
Custo	Rápida e determinística	Retomar pode demorar se houve interrupções
API lá dentro	Não (task)	Não

GESTÃO DE RECURSOS

Mutexes

Um tipo especial de semáforo binário para controlar o acesso a um recurso partilhado. Requer **configUSE_MUTEXES = 1**.

Pensar num **token** associado ao recurso: para usar o recurso, a task tem de tirar o token; no fim, tem de o devolver. Funciona por disciplina: todas as tasks aceitam só usar o recurso quando têm o token.

Exclusão mútua: o semáforo é sempre devolvido.

Sincronização (módulo 5): normalmente não é devolvido.

Exemplo 8.1: vPrintString() com um mutex

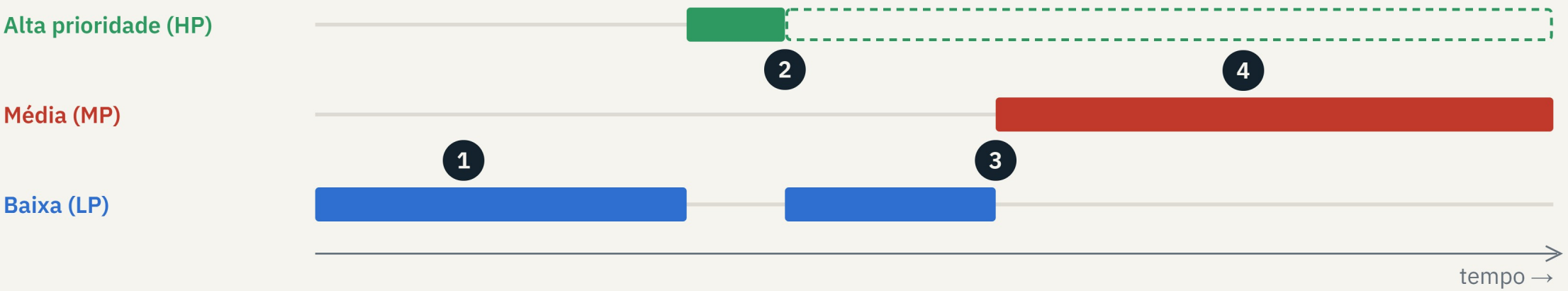
```
xMutex = xSemaphoreCreateMutex(); /* no main() */

static void prvNewPrintString( const char *pcString )
{
    xSemaphoreTake( xMutex, portMAX_DELAY );
    {
        printf( "%s", pcString );
        fflush( stdout );
    }
    xSemaphoreGive( xMutex );
}
```

xSemaphoreCreateMutex() devolve NULL sem heap; há versão Static.

GESTÃO DE RECURSOS

Priority inversion

Uma task de alta prioridade fica à espera de uma de baixa prioridade que tem o recurso. Com um lock **sem priority inheritance** (ex.: um semáforo binário), uma task de prioridade média pode impedir a de baixa de correr, **sem limite de tempo**.



1 LP tira o semáforo e é preemptada por HP.

2 HP tenta tirar o semáforo, que está com LP: bloqueia (tracejado).

3 LP continua, mas é preemptada por MP antes de devolver o semáforo.

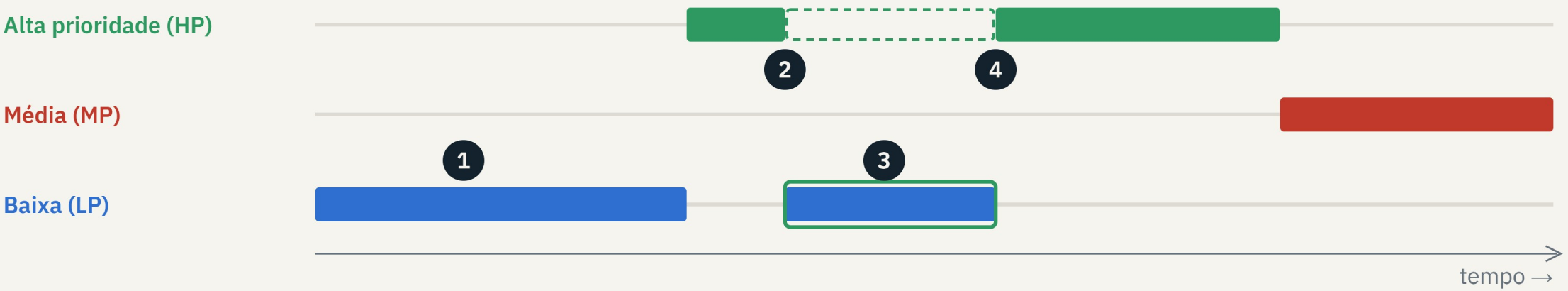
4 MP corre. HP continua à espera de LP, e LP nem sequer executa.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

GESTÃO DE RECURSOS

Priority inheritance

Os mutexes (e não os semáforos binários) sobem temporariamente a prioridade de quem tem o mutex até à da task mais prioritária que o espera. **Limita** a inversão no tempo; não a elimina.



1 LP tira o mutex e é preemptada por HP.

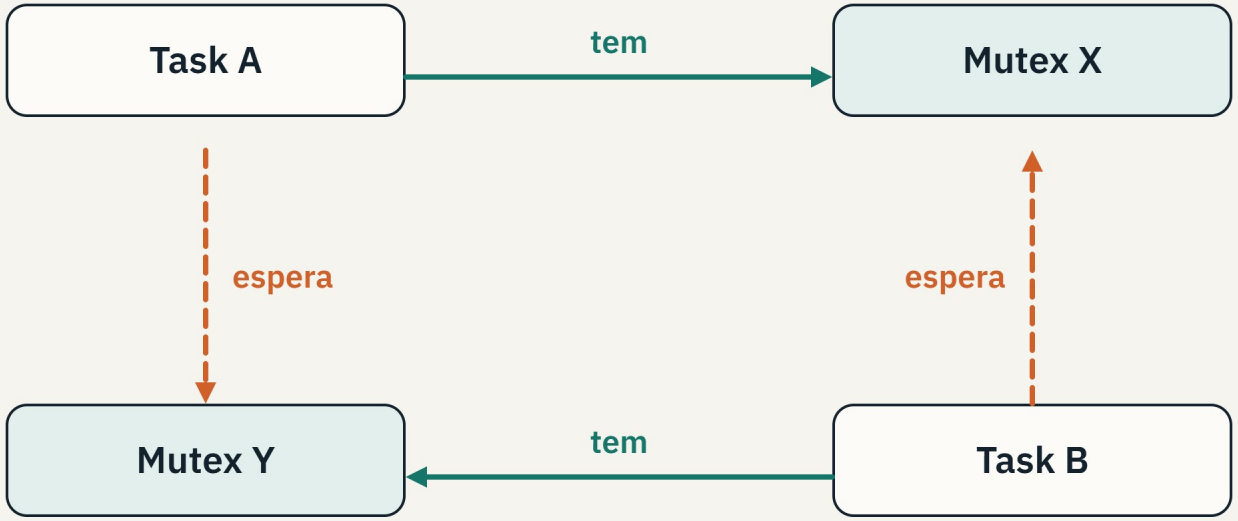
2 HP tenta tirar o mutex e bloqueia.

3 LP herda a prioridade de HP (contorno verde): MP já não a preempta.

4 LP devolve o mutex e volta à sua prioridade; HP corre. MP só corre quando HP bloqueia.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

Deadlock



Duas tasks que precisam dos mutexes X **e** Y:

1. A tira o mutex X.
2. A é preemptada por B.
3. B tira Y, tenta X: está com A, bloqueia.
4. A tenta Y: está com B, bloqueia. Nenhuma avança.

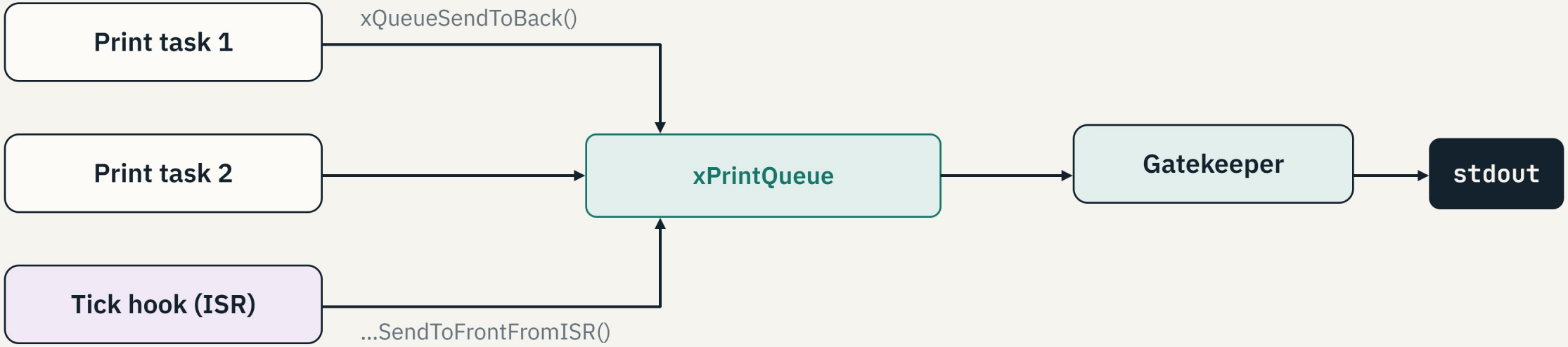
Evitar em design. Usar um timeout um pouco maior que a espera máxima prevista: falhar passa a ser sintoma de erro de design.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

GESTÃO DE RECURSOS

Gatekeeper tasks

Uma task é a **única dona** do recurso; as outras usam-no indiretamente, pedindo-lhe por uma queue. Exclusão mútua sem risco de priority inversion nem de deadlock.



Passa quase todo o tempo em Blocked. As ISRs também podem enviar para a queue: aqui, um tick hook a cada 200 ticks.

Prioridade baixa: não atrasa ninguém, mas as mensagens esperam. Alta: o contrário.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

Exemplo: gatekeeper e tick hook

A GATEKEEPER TASK

```
static void prvStdioGatekeeperTask(
                void *pvParameters )
{
    char *pcMessageToPrint;
    for( ;; )
    {
        xQueueReceive( xPrintQueue, &pcMessageToPrint,
                       portMAX_DELAY );
        printf( "%s", pcMessageToPrint );
        fflush( stdout );
    }
}
```

O tick hook · configUSE_TICK_HOOK = 1

```
void vApplicationTickHook( void )
{
    static int iCount = 0;
    if( ++iCount ≥ 200 )
    {
        xQueueSendToFrontFromISR( xPrintQueue,
                &( pcStringsToPrint[ 2 ] ), NULL );
        iCount = 0;
    }
}
```

O tick hook corre **dentro do tick interrupt**: muito curto, pouca stack, só API ...FromISR(). Como o scheduler corre logo a seguir, pode passar NULL em pxHigherPriorityTaskWoken.

GESTÃO DE RECURSOS

Qual usar?

Técnica	Protege de	Indicada para	Custo / risco
Critical section	Tasks e ISRs	Troços muito curtos	Atrasa as interrupções
Scheduler suspenso	Só tasks	Troços mais longos	Retomar pode demorar; sem API
Mutex	Tasks (nunca em ISRs)	Recurso de várias tasks	Priority inversion, deadlock
Gatekeeper task	Tasks e ISRs (via queue)	Recurso com um só dono	Mais uma task e uma queue

Antes de escolher uma: dá para desenhar o sistema de forma a que o recurso **não seja partilhado**?

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 8

MÓDULO 6 EM RESUMO

Proteger um recurso partilhado custa sempre alguma coisa. O mais barato é não o partilhar.

- Read-modify-write
- Reentrância
- taskENTER_CRITICAL
- vTaskSuspendAll
- xSemaphoreCreateMutex
- Priority inversion
- Priority inheritance
- Deadlock
- Gatekeeper

A seguir, no Módulo 7: software timers.

FORMAÇÃO · MÓDULO 7

Software timers

Gestão do tempo: executar funções no futuro ou periodicamente

O QUE SÃO

Callbacks, one-shot e auto-reload, estados Dormant e Running

COMO FUNCIONAM

Daemon task e timer command queue

A API

Criar, iniciar, timer ID, mudar o período, reset

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulo 6

O que é um software timer

Agenda a execução de uma função, a **callback**, para um momento no futuro ou periodicamente.

É implementado e controlado pelo kernel: **não usa** hardware timers nem contadores.

Não gasta tempo de processamento, exceto enquanto uma callback está a executar.

PARA ATIVAR: COMPILAR TIMERS.C E DEFINIR

configUSE_TIMERS	1 para incluir os software timers.
configTIMER_TASK_PRIORITY	Prioridade da daemon task (0 a configMAX_PRIORITIES – 1).
configTIMER_QUEUE_LENGTH	Máximo de comandos por processar na timer command queue.
configTIMER_TASK_STACK_DEPTH	Stack da daemon task, em words.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 6

A callback

```
void ATimerCallback( TimerHandle_t xTimer );
```

Uma função C normal: devolve void e recebe o handle do timer que expirou. Executa do início ao fim e sai.

Deve ser **curta** e **nunca** pode entrar em Blocked.

> Todas as callbacks correm no contexto da **daemon task**. Se uma bloquear, bloqueia a daemon task e todos os timers.

Pode

xQueueReceive() com xTicksToWait = 0.

Não pode

vTaskDelay(), ou qualquer chamada que ponha a task em Blocked.

SOFTWARE TIMERS

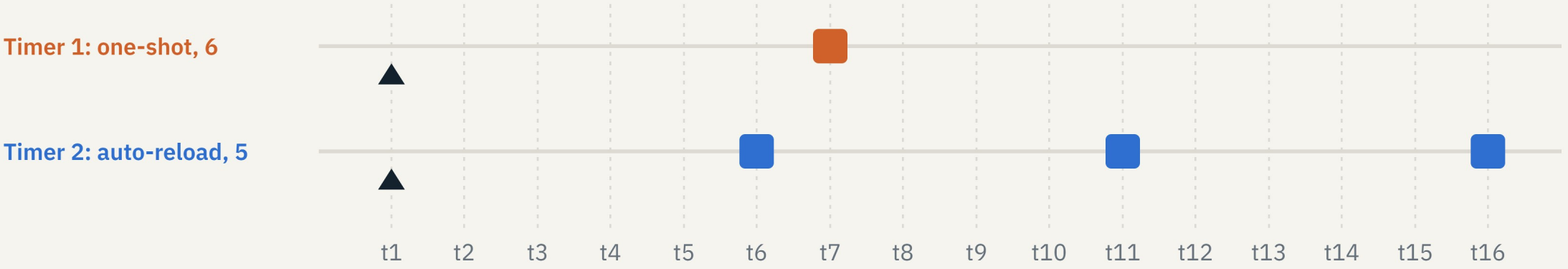
One-shot e auto-reload

ONE-SHOT

Depois de iniciado, executa a callback **uma só vez**. Pode ser reiniciado à mão, mas não se reinicia sozinho.

AUTO-RELOAD

Reinicia-se sozinho sempre que expira: a callback executa **periodicamente**.



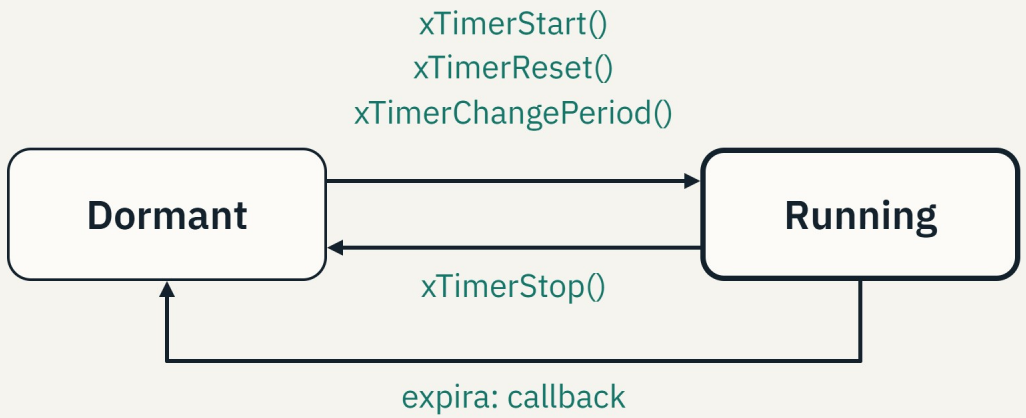
Triângulo = timer iniciado; quadrado = callback executa. Período = tempo entre o início do timer e a execução da callback.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 6

Estados: Dormant e Running

AUTO-RELOAD

ONE-SHOT



Dormant: existe e tem handle, mas não corre; a callback não executa.

Running: executa a callback quando passa o período desde que entrou em Running ou desde o último reset.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 6

A daemon task e a timer command queue

Uma task normal

Criada automaticamente com o scheduler. Prioridade e stack: configTIMER_TASK_PRIORITY e configTIMER_TASK_STACK_DEPTH.

Uma queue normal

Também criada com o scheduler. Comandos como start, stop e reset seguem por ela.

Chamava-se timer service task

Hoje também faz outras coisas, como o processamento adiado do módulo 5.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 6

SOFTWARE TIMERS

A prioridade da daemon task

A daemon task é escalonada como qualquer outra: só processa comandos ou executa callbacks quando é a task de maior prioridade pronta a correr.

DAEMON COM PRIORIDADE MAIS BAIXA

A task que chama xTimerStart() continua a correr até ao fim da função. O comando fica na queue até essa task bloquear e a daemon ser a mais prioritária.

DAEMON COM PRIORIDADE MAIS ALTA

xTimerStart() acorda a daemon task, que preempta logo a task que chamou. O comando é processado **antes** de xTimerStart() retornar.

Os comandos levam um **timestamp**: um timer de 10 ticks expira 10 ticks depois de o comando ser **enviado**, não depois de ser processado.

xTimerCreate() e xTimerStart()

```
TimerHandle_t xTimerCreate(
    const char * const pcTimerName,
    const TickType_t xTimerPeriodInTicks,
    const BaseType_t xAutoReload,
    void * const pvTimerID,
    TimerCallbackFunction_t pxCallbackFunction );

BaseType_t xTimerStart( TimerHandle_t xTimer,
                        TickType_t xTicksToWait );
```

xTicksToWait: espera por espaço na timer command queue. pdPASS se o comando foi enviado; pdFAIL se a queue estava cheia.

pcTimerName	Nome, só para debug.
xTimerPeriodInTicks	Período em ticks (pdMS_TO_TICKS). Não pode ser 0.
xAutoReload	pdTRUE: auto-reload. pdFALSE: one-shot.
pvTimerID	Valor inicial do timer ID (void *).
pxCallbackFunction	A callback.
Retorno	Handle (TimerHandle_t), ou NULL sem heap.

O timer nasce em **Dormant**. Pode ser criado e iniciado antes do scheduler: só começa a contar quando o scheduler arranca. Numa ISR: xTimerStartFromISR().

Exemplo: uma callback, dois timers, timer ID

main(): criar e iniciar

```
xOneShotTimer = xTimerCreate( "OneShot",
        pdMS_TO_TICKS( 3333 ), pdFALSE,
        NULL, prvTimerCallback );
xAutoReloadTimer = xTimerCreate( "AutoReload",
        pdMS_TO_TICKS( 500 ), pdTRUE,
        NULL, prvTimerCallback );

xTimerStart( xOneShotTimer, 0 );
xTimerStart( xAutoReloadTimer, 0 );
vTaskStartScheduler();
```

A CALLBACK: O ID GUARDA UM CONTADOR

```
static void prvTimerCallback( TimerHandle_t xTimer )
{
    uint32_t ulExecutionCount;
    ulExecutionCount =
        ( uint32_t ) pvTimerGetTimerID( xTimer );
    ulExecutionCount++;
    vTimerSetTimerID( xTimer,
                      ( void * ) ulExecutionCount );
    if( ( xTimer == xAutoReloadTimer ) &&
        ( ulExecutionCount == 5 ) )
    {
        /* Na daemon task: nunca bloquear */
        xTimerStop( xTimer, 0 );
    }
}
```

vTimerSetTimerID() e pvTimerGetTimerID() acedem ao timer diretamente, sem passar pela command queue.

xTimerChangePeriod()

```
BaseType_t xTimerChangePeriod(
    TimerHandle_t xTimer,
    TickType_t xNewPeriod,
    TickType_t xTicksToWait );
```

Timer em Running

Recalcula a expiração com o novo período, a contar do momento da chamada.

Timer em Dormant

Calcula a expiração e passa a Running: o timer arranca.

DEMOS DO FREERTOS: O CHECK TIMER PISCA UM LED

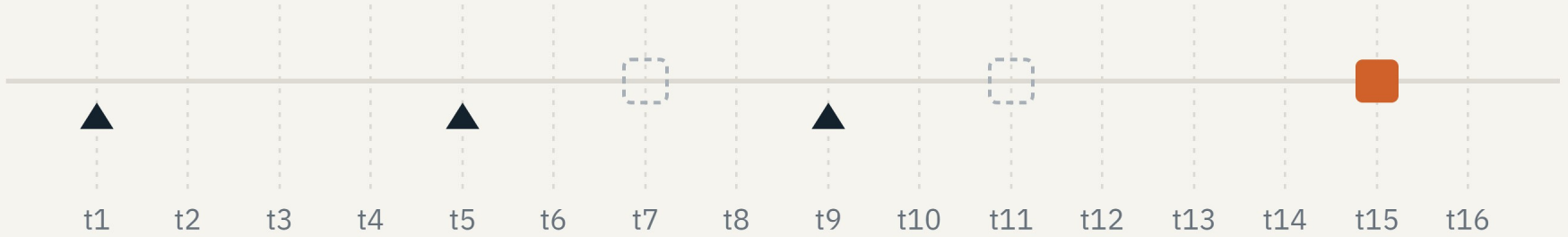
```
/* LED a cada 3 s se tudo bem; 200 ms se houver erro */
const TickType_t xErrorTimerPeriod = pdMS_TO_TICKS( 200 );
static void prvCheckTimerCallbackFunction(
        TimerHandle_t xTimer )
{
    static BaseType_t xErrorDetected = pdFALSE;
    if( xErrorDetected == pdFALSE &&
        CheckTasksAreRunningWithoutError() == pdFAIL )
    {
        xTimerChangePeriod( xTimer,
                xErrorTimerPeriod, 0 );
        xErrorDetected = pdTRUE;
    }
    ToggleLED();
}
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 6

xTimerReset()

Reiniciar um timer: a expiração passa a contar a partir do **reset**, e não do início original. Também inicia um timer em Dormant.

```
BaseType_t xTimerReset( TimerHandle_t xTimer, TickType_t xTicksToWait );
```



Iniciado em t1 (expiraria em t7), reset em t5 (t11) e em t9: expira em **t15**. Tracejado = expiração que não chegou a acontecer.

Exemplo: luz do ecrã de um telemóvel. Cada tecla liga a luz e faz reset a um one-shot; se nada for premido no período, a callback apaga a luz.

MÓDULO 7 EM RESUMO

Um software timer é um comando para a daemon task: executa uma callback curta, que nunca bloqueia, no momento certo.

Callback | One-shot | Auto-reload | Dormant · Running | Daemon task | Timer command queue | xTimerCreate / xTimerStart | Timer ID | xTimerChangePeriod | xTimerReset

A seguir: troubleshooting e ferramentas de debug.

FORMAÇÃO · MÓDULO 8

Troubleshooting e debug

Os erros mais comuns e as ferramentas para os encontrar

OS 3 ERROS MAIS COMUNS

Prioridades de interrupção, stack overflow, printf()

OUTROS SINTOMAS

Crashes no arranque, em ISRs, interrupções desligadas

AS FERRAMENTAS

configASSERT, hooks, run-time stats, trace

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), capítulos 12 e 13

TROUBLESHOOTING

Os três erros mais comuns com o FreeRTOS

01

Prioridades de interrupção

A causa número um. Com configASSERT() definido, a maioria dos ports apanha-a logo.

Na Pico 2 (RISC-V): não se aplica

02

Stack overflow

A segunda causa. Há API e verificações em run time para a apanhar.

03

printf() e sprintf()

Usar printf() para depurar pode piorar o problema que se procura.

Recomendação: ter **configASSERT()** sempre definido durante o desenvolvimento e o debug.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 13

FERRAMENTAS DE DEBUG

configASSERT()

Funciona como o assert() do C. O código do FreeRTOS chama-o em muitos pontos; a aplicação define-o no FreeRTOSConfig.h.

Uma assertion falhada é um **erro fatal**: não continuar depois dessa linha.

Custo: mais código e execução mais lenta. Sem definição, as chamadas desaparecem no pré-processador.

COM DEBUGGER: PARAR NA LINHA QUE FALHOU

```
#define configASSERT( x ) \
    if( ( x ) == 0 ) { \
        taskDISABLE_INTERRUPTS(); for( ;; ); }
```

SEM DEBUGGER: REGISTAR FICHEIRO E LINHA

```
/* FreeRTOSConfig.h */
extern void vAssertCalled( const char *pcFile,
                           unsigned long ulLine );
#define configASSERT( x ) \
    if( ( x ) == 0 ) vAssertCalled( __FILE__, __LINE__ )

/* Num ficheiro .c */
void vAssertCalled( const char *pcFile, uint32_t ulLine )
{
    RecordErrorInformationHere( pcFile, ulLine );
    taskDISABLE_INTERRUPTS();
    for( ;; );
}
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 12

TROUBLESHOOTING

01 · Prioridades de interrupção

Com nesting, uma ISR que usa a API tem de ter prioridade **igual ou inferior** (logicamente) a **configMAX_SYSCALL_INTERRUPT_PRIORITY**. Senão interrompe o kernel a meio de uma critical section: falhas intermitentes.

CORTEX-M: O ERRO DO LIVRO

- Prioridade por omissão = a mais alta (0): uma ISR que usa a API não pode ficar por configurar
- Número alto = prioridade lógica baixa: com MAX_SYSCALL = 5, a ISR pode ter 5 ou 6, nunca 3
- Bits deslocados ou não, 3 ou 4 bits por fabricante: todos para preempção
- configASSERT() apanha-o em quase todos os ports

PICO 2 (RISC-V): NESTING É OPCIONAL

- Por omissão, sem nesting: a prioridade só escolhe qual ISR pendente corre primeiro
- Com **portcontextALLOW_NESTED_IRQs = 1** e **irq_set_priority()**, uma ISR mais prioritária interrompe outra
- Mesmo assim, as critical sections desligam **todas** as interrupções: qualquer ISR pode usar FromISR, e este erro não existe
- Novos cuidados: stack das ISRs maior, dados partilhados entre ISRs; opção desligada por omissão no port

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 13; port RP2350_RISC-V

TROUBLESHOOTING

02 · Stack overflow

QUANTO SOBROU, NO PIOR MOMENTO

```
UBaseType_t uxTaskGetStackHighWaterMark(
        TaskHandle_t xTask );
```

Devolve o mínimo de stack livre desde que a task começou. Quanto mais perto de 0, mais perto do overflow. NULL = a própria task.

HOOK CHAMADO QUANDO É DETETADO

```
void vApplicationStackOverflowHook(
        TaskHandle_t xTask, char *pcTaskName );
```

Serve para apanhar e depurar; não há forma real de recuperar. Alguns MCUs dão fault antes de o hook ser chamado.

configCHECK_FOR_STACK_OVERFLOW

	Método	Custo
1	Depois de guardar o contexto numa troca de task, verifica se o stack pointer está dentro da stack.	Rápido; pode falhar overflows entre trocas.
2	Também verifica se os últimos 20 bytes da stack mantêm o padrão com que foi preenchida.	Um pouco mais lento; apanha quase todos.
3	Verifica a stack das ISRs (só alguns ports). Dispara um assert, não o hook.	Específico do port.

Os métodos aumentam o tempo de cada troca de contexto.

TROUBLESHOOTING

03 · printf() e sprintf()

Um erro frequente: o printf() causa o problema, e para o depurar acrescentam-se mais printf().

Pode não ser thread-safe

Nem adequado a ISRs, e pode demorar muito, conforme o destino do output.

Executável muito maior

Uma implementação genérica pode aumentar muito o tamanho do código.

Pode chamar malloc()

Inválido se o esquema de heap não for o heap_3.

Stack muitas vezes maior

Muito mais do que a task precisaria de outra forma.

printf-stdarg.c, usado em muitas demos: um sprintf() mínimo e económico em stack. Atenção: nem todas as cópias implementam snprintf(), e tem licença própria.

TROUBLESHOOTING

Outros sintomas comuns

Sintoma	Causa provável / o que fazer
Juntar uma task e a demo crasha	Heap à justa: vTaskStartScheduler() só retorna se faltar heap.
Crash ao usar a API numa ISR	Só ...FromISR(); nenhuma API em ISRs acima de MAX_SYSCALL.
Crash ocasional dentro de uma ISR	Stack overflow na ISR; sintaxe do port; prioridades de interrupção.
Crash ao iniciar a primeira task	Handlers do FreeRTOS não instalados; CPU sem modo privilegiado.
Interrupções ficam desligadas	API antes do scheduler desliga-as de propósito; só mexer com as macros.
Crash antes de o scheduler arrancar	ISRs a usar objetos do kernel ou a pedir trocas antes do arranque.
Crash ao usar a API em zona protegida	Nada de API com o scheduler suspenso ou numa critical section.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 13

FERRAMENTAS DE DEBUG

Estado das tasks e run-time stats

uxTaskGetSystemState()

Array de TaskStatus_t, uma por task: nome, estado, prioridade atual e base, run time, high water mark da stack...

vTaskListTasks()

A mesma informação numa tabela de texto. Estado: X Running, R Ready, B Blocked, S Suspended, D Deleted.

vTaskGetRunTimeStatistics()

Tempo em Running por task, em valor absoluto e em percentagem.

> As funções de texto são pesadas e suspendem o scheduler: **só para debug**.

NO FREERTOSCONFIG.H

```
#define configUSE_TRACE_FACILITY 1
#define configUSE_STATS_FORMATTING_FUNCTIONS 1
#define configGENERATE_RUN_TIME_STATS 1
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() \
        vSetupTimerForRunTimeStats()
#define portGET_RUN_TIME_COUNTER_VALUE() \
        ulReadRunTimeCounter()
```

Relógio **10 a 100×** mais rápido que o tick.

EXEMPLO DE OUTPUT (ILUSTRATIVO)

```
Sensor          41230     4%
Leds             8215    <1%
Uart           152870    15%
Tmr Svc          3120    <1%
IDLE           795400    79%
```

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 12

FERRAMENTAS DE DEBUG

Trace hooks, Tracealyzer e debuggers

TRACE HOOK MACROS

Macros vazias no kernel, redefinidas no FreeRTOSConfig.h para inserir código.

TRACEALYZER FOR FREERTOS

Da Percepio: captura o run time e mostra-o em mais de 20 vistas gráficas ligadas.

PLUG-INS DE DEBUGGER

Na Pico 2: **RTOS Views** do Cortex-Debug no VS Code (a testar em RISC-V).

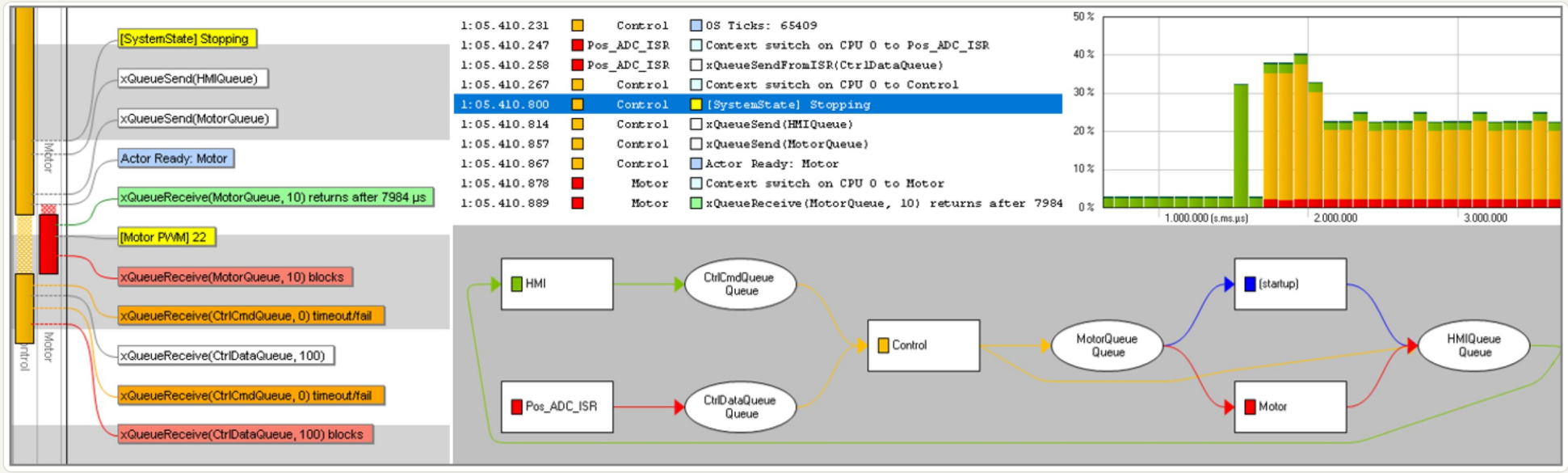


Imagem: freertos.org / Percepio Tracealyzer.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 12

TROUBLESHOOTING

Checklist quando algo corre mal

- **Definir configASSERT()**
  Apanha os erros mais comuns no ponto onde acontecem.
- **Ativar a verificação de stack**
  configCHECK_FOR_STACK_OVERFLOW = 1 ou 2, com o stack overflow hook.
- **Rever a configuração do scheduler**
  configUSE_PREEMPTION e configUSE_TIME_SLICING de acordo com a aplicação.
- **Ativar o malloc failed hook**
  configUSE_MALLOC_FAILED_HOOK = 1: saber logo quando a criação de um objeto falha.
- **Verificar os valores de retorno**
  xTaskCreate(), xQueueCreate(), xSemaphoreTake()...
- **Exceções na Pico 2 (RISC-V)**
  Numa exceção, o CPU fica parado num ciclo infinito. Pausar no debugger: **mcause** diz o tipo, **mepc** onde falhou.

Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), cap. 13

MÓDULO 8 EM RESUMO

Ativar as verificações do kernel desde o primeiro dia: é mais barato do que procurar o erro depois.

configASSERT · mcause · mepc · High water mark · configCHECK_FOR_STACK_OVERFLOW · vTaskListTasks · Run-time stats · Trace hooks · Tracealyzer

Obrigado. Perguntas?

FORMAÇÃO · SESSÃO PRÁTICA 3

Interrupções, recursos, timers e debug

O essencial, com os exemplos oficiais do livro numa Pico 2

INTERRUPÇÕES (CAP. 7)

Semáforo binário, counting semaphore e queues a partir de uma ISR

RECURSOS E TIMERS (CAPS. 8 E 6)

Mutex, gatekeeper, one-shot, auto-reload e reset

DEBUG (CAPS. 12 E 13)

configASSERT a funcionar e stack overflow

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), caps. 6, 7, 8, 12 e 13, e exemplos do repositório FreeRTOS-Kernel-Book

SESSÃO PRÁTICA 3

Os exemplos oficiais que vamos usar

Livro: *Mastering the FreeRTOS Real Time Kernel* (V1.1.0) · github.com/FreeRTOS/FreeRTOS-Kernel-Book

Exemplo do livro	O que faz	O que mostra	Exercício
6.1, 6.2	One-shot e auto-reload; timer ID	Callbacks na daemon task	6
6.3	Reset de um timer (backlight)	xTimerReset recomeça a contagem	7
7.1	Semáforo binário a partir de uma ISR	Processamento diferido	1
7.2	Counting semaphore	Não perder eventos	2
7.4	Queues a partir de uma ISR	Dados da ISR para uma task	3
8.1	Mutex a proteger a saída	Exclusão mútua	4
8.2	Gatekeeper task e tick hook	Um só dono, sem mutex	5
Caps. 12 e 13	configASSERT e stack overflow	Apanhar o erro cedo	8, 9

O mesmo repositório

Como na sessão 2. Ler o main.c de cada exemplo antes de o portar.

github.com/FreeRTOS/FreeRTOS-Kernel-Book/tree/main/examples/Win32-simulator-MSVC

O essencial

O mínimo para usar ISRs, recursos partilhados e timers sem surpresas, e para saber o que fazer quando algo pára.

Os timers (6.x) ficam para o fim: precisam da daemon task.

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

SESSÃO PRÁTICA 3

Portar os exemplos do simulador para a Pico 2

No simulador Windows	Na Pico 2
vPortGenerateSimulatedInterrupt(3)	Carregar no botão do GP0: interrupção de GPIO
vPortSetInterruptHandler(3, ulExampleInterruptHandler)	gpio_set_irq_enabled_with_callback(… vButtonISR)
vPrintString(), vPrintStringAndNumber()	Duas macros sobre o printf (RTT); no Ex. 4, uma versão lenta
_kbhit()	Um botão no GP0, ligado ao GND (pull-up interno)
printf() dentro do tick hook	Nunca: só FromISR (o Ex. 8.2 já faz assim)

```
/* Ex. 1-3 e 5-9: as funções do livro sobre o printf */
#define vPrintString( s ) printf( "%s", ( s ) )
#define vPrintStringAndNumber( s, n ) \
    printf( "%s %lu\r\n", ( s ), ( unsigned long )( n ) )
```

O Ex. 4 tem a sua vPrintString: lenta e sem proteção, de propósito.

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

SESSÃO PRÁTICA 3

A interrupção e a configuração

A INTERRUPÇÃO: O BOTÃO DO GP0

hardware extra: botão

```
#define BUTTON_PIN 0 /* ao GND, pull-up interno */

/* no main(), antes de vTaskStartScheduler() */
gpio_init( BUTTON_PIN );
gpio_set_dir( BUTTON_PIN, GPIO_IN );
gpio_pull_up( BUTTON_PIN );
gpio_set_irq_enabled_with_callback( BUTTON_PIN,
    GPIO_IRQ_EDGE_FALL, true, vButtonISR );

/* corre na interrupção do GPIO */
static void vButtonISR( uint gpio, uint32_t events )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    /* … chamadas FromISR( …, &xHigher… ) … */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

ACRESCENTAR AO FREERTOSCONFIG.H

```
#define configUSE_MUTEXES 1
#define configUSE_COUNTING_SEMAPHORES 1
#define configUSE_TICK_HOOK 1
#define configUSE_TIMERS 1
#define configTIMER_TASK_PRIORITY \
            ( configMAX_PRIORITIES - 1 )
#define configTIMER_QUEUE_LENGTH 10
#define configTIMER_TASK_STACK_DEPTH 1024
#define INCLUDE_xTimerPendFunctionCall 1
```

> Sem prioridades a acertar: neste port qualquer ISR pode usar a API FromISR, e não há nesting (módulo 5).

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 1 · EXEMPLO 7.1 hardware extra: botão

Uma ISR sincroniza uma task

FAZER

1. Handler task com prioridade 3. A interrupção vem do botão do GP0.
2. A ISR só faz give do semáforo binário.
3. Carregar no botão e ler o RTT Viewer; depois tirar o portYIELD_FROM_ISR e comparar.

PREVER ANTES DE CORRER

Quantos µs entre a ISR e o handler, com e sem portYIELD_FROM_ISR?

OBSERVAR

Com o yield: poucos µs, sempre parecido. O handler corre à saída da ISR. Sem ele: entre 0 e ~1000 µs, diferente a cada toque, porque a troca espera pelo tick seguinte. Várias linhas por toque: ressaltos do botão.

```
static SemaphoreHandle_t xBinarySemaphore;
static volatile uint32_t ulIsrTime;
/* no main(), antes de vTaskStartScheduler() */
xBinarySemaphore = xSemaphoreCreateBinary();

static void vHandlerTask( void *pvParameters )
{
    for( ;; )
    {
        xSemaphoreTake( xBinarySemaphore, portMAX_DELAY );
        uint32_t ulUs = time_us_32() - ulIsrTime;
        printf( "Handler: %u us depois da ISR\r\n",
                ( unsigned ) ulUs );
        busy_wait_ms( 500 );
    }
}

/* na vButtonISR */
ulIsrTime = time_us_32();
xSemaphoreGiveFromISR( xBinarySemaphore,
                       &xHigherPriorityTaskWoken );
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 2 · EXEMPLO 7.2 hardware extra: botão

Vários eventos antes de a task correr

FAZER

1. Primeiro: a ISR do Ex. 1 com três gives seguidos, ainda com o semáforo binário (no caso do botão ter debounce por hardware).
2. Depois: trocar por xSemaphoreCreateCounting(10, 0).
3. Contar as mensagens do handler por interrupção.

PREVER ANTES DE CORRER

Quantas vezes corre o handler por interrupção, com o binário e com o counting semaphore?

OBSERVAR

Binário: uma. O segundo e o terceiro give falham, porque o semáforo já está disponível. Counting: três (mais, se o botão ressaltar). O semáforo guarda quantos eventos faltam processar (até 10).

```
static SemaphoreHandle_t xCountingSemaphore;
/* no main(), antes de vTaskStartScheduler() */
xCountingSemaphore = xSemaphoreCreateCounting( 10, 0 );

static void vButtonISR( uint gpio, uint32_t events )
{
    BaseType_t xWoken = pdFALSE;

    /* três eventos na mesma interrupção */
    xSemaphoreGiveFromISR( xCountingSemaphore,
                           &xWoken );
    xSemaphoreGiveFromISR( xCountingSemaphore,
                           &xWoken );
    xSemaphoreGiveFromISR( xCountingSemaphore,
                           &xWoken );

    portYIELD_FROM_ISR( xWoken );
}
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 3 · EXEMPLO 7.4 hardware extra: botão

Queues a partir de uma ISR

FAZER

1. vIntegerGenerator (prio 1): a cada 200 ms envia 5 inteiros para a queue.
2. Ao carregar no botão, a ISR converte cada inteiro num ponteiro para uma string e envia-o para outra queue.
3. vStringPrinter (prio 2) imprime.

PREVER ANTES DE CORRER

Quantas strings por toque no botão?

OBSERVAR

Dez de seguida: a queue de inteiros já está cheia (o gerador envia com block time 0 e perde o resto). A queue leva ponteiros para strings constantes, não as strings.

```
static void vIntegerGenerator( void *pvParameters )
{ TickType_t xLastWake = xTaskGetTickCount();
    uint32_t ulValue = 0;
    for( ;; )
    { vTaskDelayUntil( &xLastWake, pdMS_TO_TICKS( 200 ) );
        for( int i = 0; i < 5; i++, ulValue++ )
            xQueueSendToBack( xIntegerQueue, &ulValue, 0 ); } }

static void vButtonISR( uint gpio, uint32_t events )
{ BaseType_t xWoken = pdFALSE;
    uint32_t ulNum;
    static const char *pcStrings[] = { "String 0\r\n",
        "String 1\r\n", "String 2\r\n", "String 3\r\n" };
    while( xQueueReceiveFromISR( xIntegerQueue, &ulNum, &xWoken )
           ≠ errQUEUE_EMPTY )
        xQueueSendToBackFromISR( xStringQueue,
            &pcStrings[ ulNum & 0x03 ], &xWoken );
    portYIELD_FROM_ISR( xWoken ); }

static void vStringPrinter( void *pvParameters )
{ char *pcString;
    for( ;; )
    { xQueueReceive( xStringQueue, &pcString, portMAX_DELAY );
        vPrintString( pcString ); } }
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 4 · EXEMPLO 8.1

Um mutex a proteger a saída

FAZER

1. Duas tasks com a mesma prvPrintTask; cada uma recebe em pvParameters uma string grande: "Task 1 ****..." (prio 1) e "Task 2 ----..." (prio 2).
2. Primeiro sem mutex: chamam a vPrintString lenta diretamente.
3. Depois com xSemaphoreCreateMutex().

PREVER ANTES DE CORRER

Sem o mutex, as linhas saem inteiras?

OBSERVAR

Sem mutex, a Task 2 interrompe a Task 1 a meio da linha e os caracteres misturam-se. Com mutex, as linhas saem inteiras e a Task 1 herda a prioridade da Task 2.

```
static SemaphoreHandle_t xMutex;
/* no main(): xMutex = xSemaphoreCreateMutex(); */

/* só neste exercício: lenta e sem proteção */
static void vPrintString( const char *pcString )
{ for( ; *pcString != 0; pcString++ )
    { putchar_raw( *pcString );
        busy_wait_us( 100 ); /* periférico lento */ } }

static void prvPrintTask( void *pvParameters )
{ const char *pcString = ( const char * ) pvParameters;
    for( ;; )
    { xSemaphoreTake( xMutex, portMAX_DELAY );
        vPrintString( pcString );
        xSemaphoreGive( xMutex );
        vTaskDelay( pdMS_TO_TICKS( 2 ) ); /* < 1 linha */ } }
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 5 · EXEMPLO 8.2

Uma gatekeeper task

FAZER

1. Queue de 5 ponteiros; só a gatekeeper (prio 1) escreve na saída.
2. Duas prvPrintTask (prio 2 e 3) recebem em pvParameters o índice da sua string e enviam-na para o fim da queue.
3. O tick hook, a cada 200 ticks, envia outra para a frente.

PREVER ANTES DE CORRER

A gatekeeper tem a prioridade mais baixa. As linhas misturam-se? A do tick hook atrasa?

OBSERVAR

Linhas sempre inteiras, sem mutex. A do tick hook sai a cada 200 ms, à frente das outras. Com a queue cheia, a print task perde a mensagem (block time 0).

```
static QueueHandle_t xPrintQueue;
/* no main(): xQueueCreate( 5, sizeof( char * ) ) */
static char *pcStringsToPrint[] = { "Task 1 ****…\r\n",
    "Task 2 ----…\r\n", "Tick hook ####…\r\n" };

static void prvPrintTask( void *pvParameters )
{ int iIndex = ( int ) pvParameters; /* 0 ou 1 */
    for( ;; )
    { xQueueSendToBack( xPrintQueue,
            &pcStringsToPrint[ iIndex ], 0 );
        vTaskDelay( pdMS_TO_TICKS( 10 ) ); } }

static void prvStdioGatekeeperTask( void *pv)
{ char *pcMsg;
    for( ;; )
    { xQueueReceive( xPrintQueue, &pcMsg, portMAX_DELAY );
        vPrintString( pcMsg ); } }

void vApplicationTickHook( void )
{ static int iCount = 0;
    if( ++iCount ≥ 200 )
    { xQueueSendToFrontFromISR( xPrintQueue,
            &pcStringsToPrint[ 2 ], NULL );
        iCount = 0; } }
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 6 · EXEMPLOS 6.1 E 6.2

One-shot, auto-reload e timer ID

FAZER

1. um one-shot de 3333 ms e um auto-reload de 500 ms, criados e arrancados antes do scheduler.
2. um só callback para os dois; o timer ID conta as execuções.
3. O auto-reload pára-se a si próprio à 5.ª execução.

PREVER ANTES DE CORRER

Em que ticks aparecem as mensagens? O one-shot repete?

OBSERVAR

Auto-reload aos 500, 1000, 1500, 2000 e 2500 ms, e pára. One-shot uma vez, aos 3333 ms. Os callbacks correm na daemon task: nunca bloquear dentro deles.

```
static TimerHandle_t xOneShotTimer, xAutoReloadTimer;

static void prvTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow = xTaskGetTickCount();
    uint32_t ulCount;
    ulCount = ( uint32_t )
              pvTimerGetTimerID( xTimer );
    ulCount++;
    vTimerSetTimerID( xTimer, ( void * ) ulCount );
    if( xTimer == xOneShotTimer )
        vPrintStringAndNumber( "One-shot", xTimeNow );
    else
    {
        vPrintStringAndNumber( "Auto-reload",
                               xTimeNow );
        if( ulCount == 5 )
            xTimerStop( xTimer, 0 );
    }
}
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 7 · EXEMPLO 6.3 hardware extra: botão

Reset de um timer: a backlight

FAZER

1. Timer one-shot de 5000 ms; o callback apaga o LED (GP1).
2. vKeyHitTask lê o botão do GP0 a cada 50 ms; se premido, acende o LED e faz xTimerReset().
3. Carregar uma vez; depois várias vezes seguidas.

PREVER ANTES DE CORRER

Se carregarmos no botão a cada 2 s, quando se apaga o LED?

OBSERVAR

Nunca, enquanto se carregar com menos de 5 s de intervalo: cada reset recomeça a contagem. Apaga 5 s depois da última vez. Num timer parado, xTimerReset() também o arranca.

```
#define BUTTON_PIN 0 /* ao GND, pull-up interno */
#define LED_PIN 1
static TimerHandle_t xBacklightTimer;

static void prvBacklightTimerCallback( TimerHandle_t xTimer )
{ gpio_put( LED_PIN, 0 ); /* apaga */
    vPrintStringAndNumber( "Backlight OFF, tick",
                           xTaskGetTickCount() ); }

static void vKeyHitTask( void *pvParameters )
{ const TickType_t xShortDelay = pdMS_TO_TICKS( 50 );
    for( ;; )
    { if( gpio_get( BUTTON_PIN ) == 0 ) /* premido */
        { gpio_put( LED_PIN, 1 );
            xTimerReset( xBacklightTimer, xShortDelay ); }
        vTaskDelay( xShortDelay ); } }
```

EXERCÍCIO 8 · SECÇÃO 12.2

Um configASSERT a funcionar

FAZER

1. Build Debug; configASSERT como no exemplo oficial, com assert().
2. Comentar o xQueueCreate e correr.
3. Ler a mensagem no RTT Viewer e a stack de chamadas no debugger.

PREVER ANTES DE CORRER

Onde pára o programa? E num build Release?

OBSERVAR

Pára logo no primeiro xQueueSend, com uma mensagem do assert com o ficheiro (queue.c) e a linha. Em Release, o NDEBUG desliga o assert: o erro passa em silêncio.

```
/* FreeRTOSConfig.h (pico-examples/freertos) */
#include <assert.h>
#define configASSERT( x ) assert( x )

/* main.c: esquecer de criar a queue */
static QueueHandle_t xQueue = NULL;
/* xQueue = xQueueCreate( 5, sizeof( uint32_t ) ); */

static void vSenderTask( void *pvParameters )
{
    uint32_t ulValue = 0;
    for( ;; )
    {
        xQueueSend( xQueue, &ulValue, 0 ); /* assert */
        vTaskDelay( pdMS_TO_TICKS( 100 ) );
    }
}
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

EXERCÍCIO 9 · SECÇÃO 13.3

Provocar e apanhar um stack overflow

FAZER

1. Com configCHECK_FOR_STACK_OVERFLOW = 2 (sessão 1), criar a vGreedyTask ao lado.
2. Correr: onde pára? Ver pcTaskName no hook.
3. Corrigir: subir a stack e medir com uxTaskGetStackHighWaterMark(NULL).

PREVER ANTES DE CORRER

O hook apanha o overflow no momento em que acontece?

OBSERVAR

Não: a verificação é feita na troca de contexto, depois do estrago. Pode aparecer primeiro uma exceção ou dados corrompidos. O high water mark mostra a margem que sobra, em words.

```
static void vGreedyTask( void *pvParameters )
{
    volatile uint8_t ucBuffer[ 600 ];
    for( ;; )
    {
        for( int i = 0; i < 600; i++ )
        {
            ucBuffer[ i ] = ( uint8_t ) i;
        }
        vTaskDelay( pdMS_TO_TICKS( 100 ) );
    }
}

/* 128 words = 512 bytes: não chega */
xTaskCreate( vGreedyTask, "Greedy", 128,
             NULL, 1, NULL );
```

Fonte: Livro, caps. 6, 7, 8, 12 e 13; exemplos Win32 013–021, adaptados à Pico 2

SESSÃO PRÁTICA 3

Perguntas para fechar

Porque é que o trabalho vai para uma task e não fica na ISR?	Ex. 1
O que se perde com um semáforo binário quando chegam vários eventos?	Ex. 2
Porque é que a queue do Ex. 3 leva ponteiros e não as strings?	Ex. 3
Mutex ou gatekeeper: quando usar cada um?	Ex. 4, 5
Em que task corre um timer callback, e o que não pode fazer?	Ex. 6, 7
Porque é que o configASSERT não pode ser o assert() num build de produção?	Ex. 8

SESSÃO PRÁTICA 3 EM RESUMO

A ISR sinaliza, uma task processa; um recurso partilhado tem um dono de cada vez; e o configASSERT diz onde está o erro.

FromISR + portYIELD_FROM_ISR | Binário vs counting | Queues a partir de ISRs | Mutex | Gatekeeper | Daemon task | xTimerReset | configASSERT | Interrupção de GPIO | High water mark

Extra: a sessão 4 aprofunda cada um destes temas.

FORMAÇÃO · SESSÃO PRÁTICA 4 EXTRA

Aprofundar: ISRs, recursos, timers e debug

Exercícios mais longos, com exemplos do livro e ideias das demos oficiais

INTERRUPÇÕES A SÉRIO

Daemon task, o botão com ressaltos e a handler task robusta

MEDIR E TESTAR

Run-time stats e um teste automático com check timer

QUANDO CORRE MAL

Priority inversion e deadlock

Rúben Marques · Fonte: Mastering the FreeRTOS Real Time Kernel (V1.1.0), caps. 6, 7, 8 e 12

SESSÃO PRÁTICA 4 · EXTRA

Os exemplos oficiais que vamos usar

Livro: *Mastering the FreeRTOS Real Time Kernel* (V1.1.0) · github.com/FreeRTOS/FreeRTOS-Kernel-Book

Exemplo do livro	O que faz	O que mostra	Exercício
7.3	Função diferida na daemon task	A prioridade da daemon	1
Listing 7.10	Botão real com handler task	Ressaltos e debounce	2
Listings 12.10 a 12.12	Run-time stats	Quem gasta o CPU	3
—	Check timer e tasks que se testam	Teste automático com um LED	4
Figuras 8.4 e 8.5	Semáforo binário vs. mutex	Priority inversion	5
Secção 8.3.4	Dois mutexes por ordem inversa	Deadlock	6

O mesmo repositório

Como nas sessões 2 e 3. Só o exemplo 7.3 é novo; o resto do código está nos slides.

github.com/FreeRTOS/FreeRTOS-Kernel-Book/tree/main/examples/Win32-simulator-MSVC

O essencial

Medir o CPU, testar o sistema e reconhecer os erros clássicos de concorrência na placa.

Fonte: Livro, caps. 6, 7, 8 e 12; adaptado à Pico 2

EXERCÍCIO 1 · EXEMPLO 7.3 hardware extra: botão

Processamento diferido na daemon task

FAZER

1. Botão no GP0 (prática 3): a ISR pede à daemon task que corra a função. Sem handler task nem semáforo.
2. vPeriodicTask abaixo da daemon: 1 s a trabalhar, 1 s bloqueada.
3. Depois: dar-lhe mais prioridade do que a daemon (configTIMER_TASK_PRIORITY + 1).

PREVER ANTES DE CORRER

Em que task corre vDeferredHandlingFunction? E se uma task mais prioritária do que a daemon estiver a trabalhar?

OBSERVAR

Corre na daemon task, logo à saída da ISR. Se a vPeriodicTask tiver mais prioridade, a mensagem espera que ela bloqueie (até 1 s). Poupa uma task, mas os eventos ficam todos à prioridade da daemon.

```
/* FreeRTOSConfig.h */
#define configUSE_TIMERS 1
#define INCLUDE_xTimerPendFunctionCall 1
#define configTIMER_TASK_PRIORITY ( configMAX_PRIORITIES - 2 )

static void vDeferredHandlingFunction( void *pvParameter1,
                                       uint32_t ulParameter2 )
{ vPrintStringAndNumber( "Evento", ulParameter2 ); }

static void vButtonISR( uint gpio, uint32_t events )
{ static uint32_t ulCount = 0;
    BaseType_t xWoken = pdFALSE;
    xTimerPendFunctionCallFromISR( vDeferredHandlingFunction,
                                   NULL, ulCount++, &xWoken );
    portYIELD_FROM_ISR( xWoken ); }

static void vPeriodicTask( void *pvParameters )
{ for( ;; )
    { busy_wait_ms( 1000 ); /* a trabalhar */
        vTaskDelay( pdMS_TO_TICKS( 1000 ) ); } } /* bloqueada */

/* no main(), antes de vTaskStartScheduler() */
gpio_set_irq_enabled_with_callback( BUTTON_PIN,
    GPIO_IRQ_EDGE_FALL, true, vButtonISR );
xTaskCreate( vPeriodicTask, "Periodic", 512, NULL,
    configTIMER_TASK_PRIORITY - 1, NULL ); /* passo 3: + 1 */
```

Fonte: Livro, cap. 7 (Exemplo 7.3); adaptado à Pico 2

EXERCÍCIO 2 · LISTING 7.10

Um botão real, com ressaltos

FAZER

1. Botão no GP0 (pull-up interno) com interrupção no flanco descendente.
2. Carregar 20 vezes e comparar ulIsrCount com ulButtonPressCounts no debugger.
3. Trocar para counting semaphore; depois filtrar os ressaltos com um one-shot e xTimerResetFromISR().

PREVER ANTES DE CORRER

Os dois contadores ficam iguais? E algum fica igual a 20?

OBSERVAR

Com o binário, ulButtonPressCounts fica abaixo de ulIsrCount: os ressaltos geram vários flancos antes de a task correr. O counting semaphore não perde nenhum, mas conta os ressaltos. Só o debounce dá 20.

```
static SemaphoreHandle_t xButtonSem;
static volatile uint32_t ulIsrCount, ulButtonPressCounts;

static void vButtonCallback( uint gpio, uint32_t events )
{ BaseType_t xWoken = pdFALSE;
    ulIsrCount++;
    xSemaphoreGiveFromISR( xButtonSem, &xWoken );
    portYIELD_FROM_ISR( xWoken ); }

/* handler task: trata o toque fora da ISR */
static void vButtonHandlerTask( void *pvParameters )
{ const TickType_t xMaxBlock = pdMS_TO_TICKS( 5000 );
    for( ;; )
    { if( xSemaphoreTake( xButtonSem, xMaxBlock ) == pdPASS )
            ulButtonPressCounts++;
        else
            vPrintString( "Sem toques em 5 s\r\n" ); } }

/* no main(), antes de vTaskStartScheduler() */
xButtonSem = xSemaphoreCreateBinary();
xTaskCreate( vButtonHandlerTask, "Botao", 512, NULL, 3, NULL );
gpio_set_irq_enabled_with_callback( BUTTON_PIN,
    GPIO_IRQ_EDGE_FALL, true, &vButtonCallback );
```

Fonte: Livro, caps. 6 e 7; adaptado à Pico 2

EXERCÍCIO 3 · LISTINGS 12.10 A 12.12

Quem gasta o CPU?

FAZER

1. Relógio: o timer do RP2350, que já conta microssegundos (time_us_64).
2. A prvStatsTask imprime a cada 5 s vTaskGetRunTimeStatistics() e vTaskListTasks().
3. Juntar ao projeto do Ex. 4 da prática 3: as duas prvPrintTask com a vPrintString lenta.

PREVER ANTES DE CORRER

Que percentagem tem a IDLE? E as tasks que escrevem com a vPrintString lenta?

OBSERVAR

A IDLE é o CPU livre. Quem escreve com espera ativa (busy_wait_us) gasta CPU só à espera: argumento para DMA ou interrupções. vTaskListTasks mostra estado, prioridade e high water mark.

```
/* FreeRTOSConfig.h */
#define configGENERATE_RUN_TIME_STATS 1
#define configUSE_TRACE_FACILITY 1
#define configUSE_STATS_FORMATTING_FUNCTIONS 1
#define configRUN_TIME_COUNTER_TYPE uint64_t
uint64_t ullGetRunTimeCounter( void );
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS()
#define portGET_RUN_TIME_COUNTER_VALUE() ullGetRunTimeCounter()

/* main.c: o timer do RP2350, a 1 MHz */
uint64_t ullGetRunTimeCounter( void ) { return time_us_64(); }

static void prvStatsTask( void *pvParameters )
{ static char cBuffer[ 512 ];
    for( ;; )
    { vTaskDelay( pdMS_TO_TICKS( 5000 ) );
        vTaskGetRunTimeStatistics( cBuffer, sizeof( cBuffer ) );
        printf( "\r\nTask Tempo(us) %%CPU\r\n%s", cBuffer );
        vTaskListTasks( cBuffer, sizeof( cBuffer ) );
        printf( "\r\nTask Estado Prio Stack Num\r\n%s", cBuffer ); } }

/* no main(), antes de vTaskStartScheduler() */
xTaskCreate( prvStatsTask, "Stats", 1024, NULL, 1, NULL );
```

Fonte: Livro, cap. 12 (secção 12.5); adaptado à Pico 2

EXERCÍCIO 4 · SECÇÃO 6.7

Um teste automático ao sistema, com um LED

FAZER

1. Produtor (prio 1) envia um número a cada 10 ms; consumidor (prio 2) confirma a sequência.
2. Um check timer de 1 s confirma que os contadores avançam e que não houve erro.
3. Depois, partir algo: uma task a configMAX_PRIORITIES − 2 que nunca bloqueia.

PREVER ANTES DE CORRER

O que faz o LED quando está tudo bem? E com um erro?

OBSERVAR

Pisca a cada 1 s. Com a task que não bloqueia, os contadores param; o check timer corre na daemon, que está acima dessa task, deteta e o LED pisca a cada 200 ms.

```
static QueueHandle_t xQ;
static volatile uint32_t ulTxCount, ulRxCount;
static volatile BaseType_t xError = pdFALSE;
static void vProducerTask( void *pv ) /* prio 1 */
{ uint32_t ulValue = 0;
    for( ;; )
    { xQueueSend( xQ, &ulValue, portMAX_DELAY );
        ulValue++; ulTxCount++;
        vTaskDelay( pdMS_TO_TICKS( 10 ) ); } }
static void vConsumerTask( void *pv ) /* prio 2 */
{ uint32_t ulExpected = 0, ulValue;
    for( ;; )
    { xQueueReceive( xQ, &ulValue, portMAX_DELAY );
        if( ulValue ≠ ulExpected++ ) xError = pdTRUE;
        ulRxCount++; } }
static void prvCheckTimerCallback( TimerHandle_t xT )
{ static uint32_t ulLastTx, ulLastRx;
    if( ulTxCount == ulLastTx || ulRxCount == ulLastRx || xError )
        xTimerChangePeriod( xT, pdMS_TO_TICKS( 200 ), 0 ); /* erro */
    ulLastTx = ulTxCount; ulLastRx = ulRxCount;
    gpio_xor_mask( 1u << LED_PIN ); }

/* no main(): a queue, as duas tasks e o check timer de 1 s */
xQ = xQueueCreate( 5, sizeof( uint32_t ) );
xTimerStart( xTimerCreate( "Check", pdMS_TO_TICKS( 1000 ),
             pdTRUE, NULL, prvCheckTimerCallback ), 0 );
```

Fonte: Livro, cap. 6 (secção 6.7); adaptado à Pico 2

EXERCÍCIO 5 · SECÇÕES 8.3.2 E 8.3.3

Priority inversion: semáforo vs. mutex

FAZER

1. Três tasks: LP (prio 1), MP (2) e HP (3).
2. Primeiro com um semáforo binário, depois com um mutex.
3. HP mede com time_us_32() quanto tempo espera pelo recurso e imprime-o.

PREVER ANTES DE CORRER

Com o semáforo binário, quem atrasa HP? E com o mutex?

OBSERVAR

Binário: HP espera por LP, e LP não corre enquanto MP corre: inversão sem limite. Mutex: LP herda a prioridade de HP, MP não a interrompe, e HP espera só o resto do trabalho de LP.

```
/* LP (1): tira o recurso e trabalha ~20 ms */
xSemaphoreTake( xLock, portMAX_DELAY );
vBusyWaitMs( 20 );
xSemaphoreGive( xLock );

/* MP (2): acorda a meio e trabalha ~50 ms */
/* HP (3): acorda a seguir e tenta o recurso */
xSemaphoreTake( xLock, portMAX_DELAY );

/* Correr duas vezes: */
xLock = xSemaphoreCreateBinary(); /* +Give inicial */
xLock = xSemaphoreCreateMutex();
```

Fonte: Livro, cap. 8 (Figuras 8.4 e 8.5); adaptado à Pico 2

EXERCÍCIO 6 · SECÇÃO 8.3.4

Provocar um deadlock

FAZER

1. Duas tasks e dois mutexes, tirados por ordem inversa.
2. Correr e abrir a vista das tasks (RTOS Views).
3. Corrigir: a mesma ordem em todo o lado, ou um block time finito com recuperação.

PREVER ANTES DE CORRER

Falha algum assert? Como se descobre o problema?

OBSERVAR

Nada falha: as duas tasks ficam Blocked para sempre, cada uma à espera do mutex da outra. Só o debugger ou um watchdog o revelam.

```
static void vTaskA( void *pvParameters )
{
    for( ;; )
    {
        xSemaphoreTake( xMutexUart, portMAX_DELAY );
        vTaskDelay( 1 ); /* abre a janela */
        xSemaphoreTake( xMutexI2C, portMAX_DELAY );
        /* … usar os dois recursos … */
        xSemaphoreGive( xMutexI2C );
        xSemaphoreGive( xMutexUart );
    }
}

/* vTaskB: igual, mas tira primeiro o I2C */
```

Fonte: Livro, cap. 8 (secção 8.3.4); adaptado à Pico 2

SESSÃO PRÁTICA 4 EM RESUMO

Medir antes de otimizar, testar depois de mudar a configuração, e desenhar para que o deadlock não seja possível.

xTimerPendFunctionCallFromISR | Ressaltos e debounce | Take com timeout | Run-time stats | vTaskListTasks

Teste automático | Check timer | Priority inheritance | Deadlock

As demos oficiais continuam a ser a melhor referência depois da formação.